Cribado fuera de muestra

AlgoProject — manual de uso. Actualizado el 2026-10-14 con `tools/manual.py`.

1. 1. Análisis IS/OOS — qué métrica del histórico predice el futuro
2. 2. Barrido de filtros — cuánto te da de verdad filtrar, y a cambio de qué
3. 3. Comparar muestras — ¿se cumplen las conclusiones en otra generación?
4. 4. Decaimiento — cuánto del edge sobrevive fuera de muestra, y si te la quedas o no
5. 29. La puerta OOS — de un databank entero a una lista de supervivientes
6. 98. Criba del MAE profundo — de qué operaciones vive una estrategia
7. 99. Monos por la cadena — ¿cuántas sin ventaja se cuelan, y cuántas buenas se pierden?
8. 108. Lotes de una celda — ¿puedo sumar varios builds como si fuesen uno grande?

1. Análisis IS/OOS — qué métrica del histórico predice el futuro

Qué pregunta responde

Cuando SQX genera miles de estrategias, todas se ven bien en el periodo con el que se construyeron (**IS**, *in-sample*, el histórico). La pregunta es cuáles siguen viéndose bien en el periodo que no vieron nunca (**OOS**, *out-of-sample*, el futuro simulado).

Este análisis coge todas las estrategias de un databank y mide, métrica por métrica, **cuánta información sobre el OOS hay en el IS**. Si el Sharpe del histórico no dice nada del futuro, filtrar por Sharpe al generar es tirar el tiempo. Si dice algo, dice cuánto.

Cuándo lo usas, y cuándo no

Lo usas cuando tienes un databank grande de estrategias generadas y quieres decidir con qué criterio filtrarlas — o con qué criterio configurar la generación en SQX para el siguiente activo.

No lo usas si las estrategias del databank ya fueron seleccionadas por su resultado OOS. Ahí el análisis mide tu propia selección, no la realidad, y sale un resultado bonito y falso. Necesita estrategias generadas **al azar** y sin filtrar por el resultado que estás estudiando.

Tampoco sirve para elegir una estrategia concreta. Habla de la población entera.

Antes de empezar

Necesitas dos cosas:

1. **Un databank con estrategias** en el proyecto de SQX, retesteado, con periodo IS y OOS. Las columnas son las de tu vista `Export Data View`, pero ya no se exportan a través de SQX: se leen del `SQStats` de cada `.sqx` (comprobado igual a la exportación de la vista). No hace falta que ningún SQX esté abierto ni cerrado.

Cómo se ejecuta

Son dos comandos. El primero saca los datos de SQX; el segundo los analiza.

```
cd ~/Desktop/AlgoProject

# 1. refrescar el CSV  (medio segundo por cada 100 estrategias: se lee de los .sqx)
python3 -m sqx.export.export_metrics --project XAUUSD --databank OOS

# 2. analizar y generar el reporte  (~1 segundo, no toca SQX)
python3 -m studies.screening.isOos.report --project XAUUSD --databank OOS
```

El paso 1 solo cuando quieras datos nuevos. El paso 2 lo puedes repetir las veces que quieras.

`export_metrics.py`

flag	obligatorio	qué hace
`--project`	sí	nombre del proyecto tal y como aparece en SQX. Sin espacios: guiones bajos
`--databank`	sí	nombre del databank, p. ej. `OOS`
`--role`	no	`conductor` o `custodian` si el proyecto vive en un worker; sin él, el maestro

⚠ **Borra la exportación anterior de ese databank antes de escribir la nueva.** A propósito: así solo hay un CSV y nunca te preguntas cuál es el bueno. Lo que sí se conserva son los reportes.

`studies.screening.isOos.report`

flag	obligatorio	qué hace
`--project`	sí	el mismo proyecto del paso 1
`--databank`	sí	el mismo databank del paso 1
`--set`	no	cambia un ajuste de `config.yaml` para esta vez, p. ej. `--set trades.bins=60`

No toca SQX. Lee lo que haya para ese databank y hace la parte que puede:

- **el CSV de métricas** del paso 1 → la parte de **población** (todo lo que sigue hasta «Un ejemplo completo»);
- **la cosecha más reciente** de la puerta (`harvest/<proyecto>/<databank>/<día>/` , la hace `/oos-gate`) → la parte **por operación**, una página por estrategia (ver «IS contra OOS, operación por operación» más abajo).

Si no hay ni lo uno ni lo otro, se para y te dice qué comando falta. Las dos partes caen en la misma carpeta y el `manifest.json` nombra las dos entradas.

Qué produce

```
~/Desktop/AlgoData/reports/XAUUSD/OOS/2026-09-04/isOos/
    explorer.html    el panel interactivo. Lo abres con doble clic
    isOos.html       las conclusiones, la misma página que pinta la ventana
    isOos.md         las conclusiones escritas, en texto
    isOos.json       el resultado entero, en el contrato de estudios
    estrategias/     una página (.html) y un resultado (.json) por estrategia: IS contra OOS
                     operación por operación — solo si el databank tiene cosecha
    manifest.json    de qué exportación y de qué cosecha salió, y de qué fecha
```

Hasta el 25-09 estos archivos caían sueltos en la carpeta del día y el texto se llamaba `summary.md` ; ahora cada estudio tiene su subcarpeta (ver `docs/MAPA-DE-CARPETAS.md`).

Una carpeta por día. Los reportes viejos no se borran nunca.

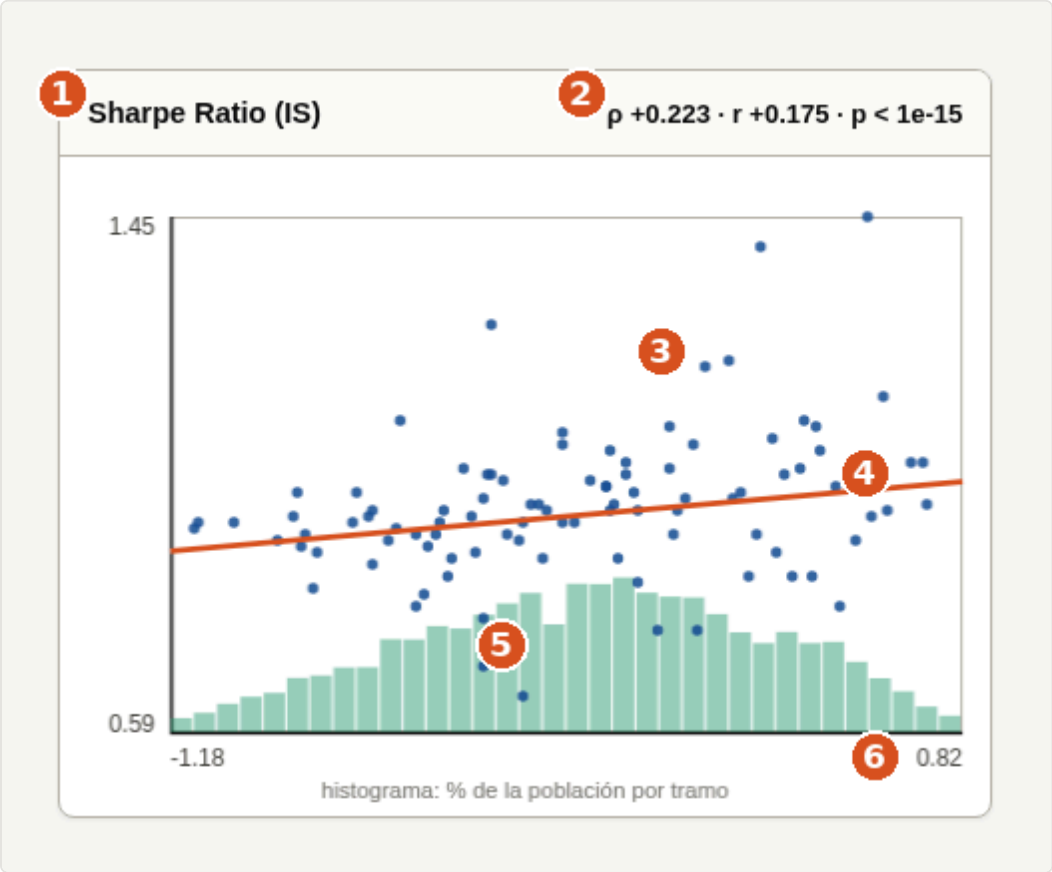
```
xdg-open ~/Desktop/AlgoData/reports/XAUUSD/OOS/2026-09-04/isOos/explorer.html
```

Cómo se lee el panel



Arriba están los controles; abajo, un gráfico por cada métrica del IS. **Los gráficos se reordenan solos**: el de arriba a la izquierda es siempre el que más relación tiene con lo que has puesto en el eje Y.

Anatomía de un gráfico



	qué es
1	la métrica del **IS** que va en el eje X. La del eje Y es la que elegiste arriba, igual en todos
2	la fuerza de la relación. **ρ (rho)** es la que manda; va de −1 a +1 y **0 significa que no hay relación**. `r` es lo mismo calculado de otra forma: si las dos se parecen, fíate; si no, hay valores extremos distorsionando. `p` es la probabilidad de ver esto por pura casualidad — cuanto más pequeño, mejor
3	cada punto es una estrategia
4	la línea de tendencia. Si sube, más de esa métrica en el IS va con mejor resultado OOS
5	**el histograma: dónde están de verdad las estrategias.** Aquí casi todas están amontonadas a la izquierda, así que esa línea de tendencia la está estirando un puñado de puntos de la derecha. Sin el histograma no lo verías
6	los extremos del eje

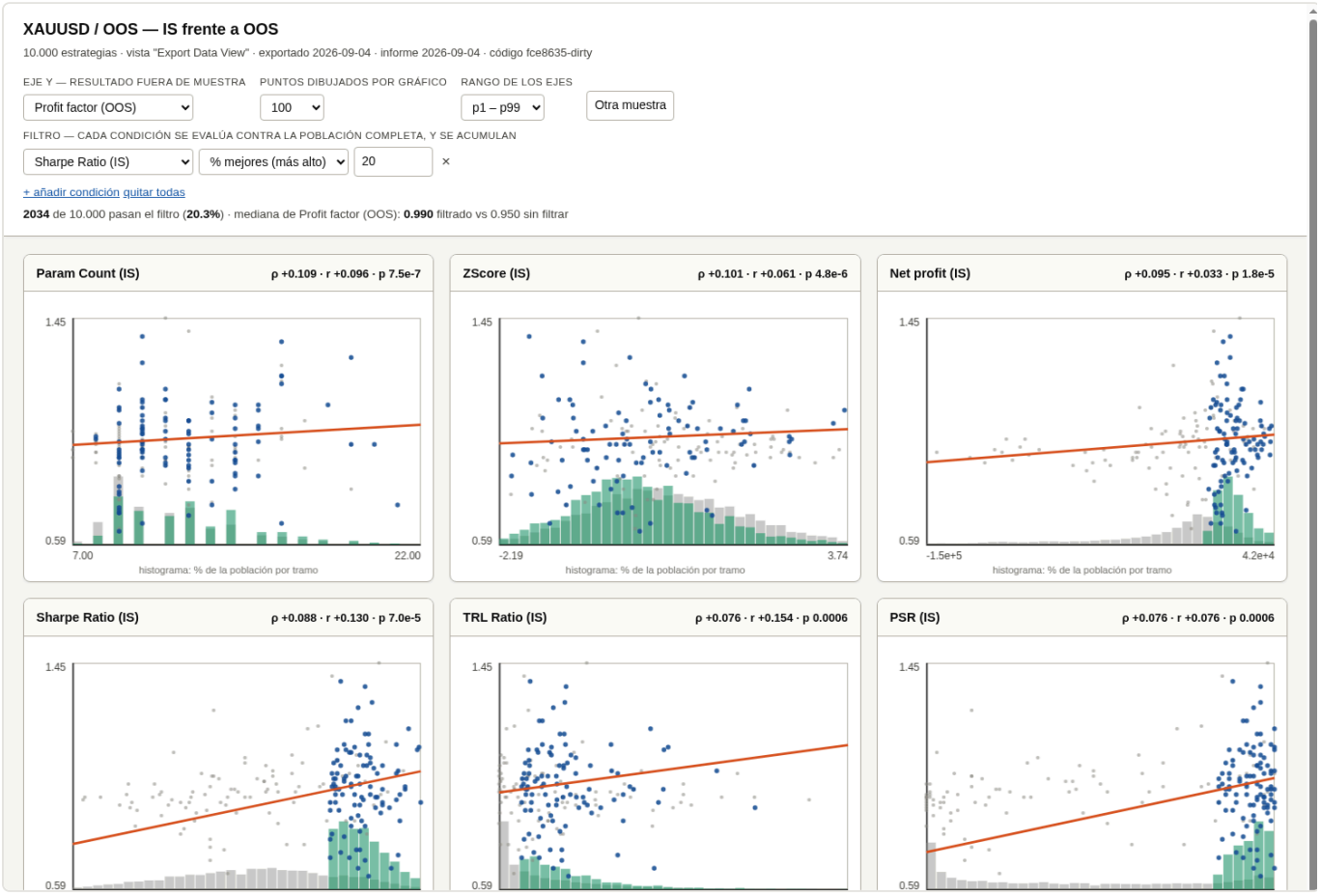
Un ρ de 0,2 es una relación real pero floja. Mueve las probabilidades, no decide el resultado. No esperes 0,8: en este negocio no existe.

Los controles

control	qué hace
Eje Y	cambia la métrica de resultado OOS que estás intentando predecir
Puntos dibujados por gráfico	cuántos puntos se **dibujan**, por defecto 100. Dibujar 10.000 puntos tapa el gráfico entero. **Los números no cambian**: ρ, la línea y el histograma siempre usan todas las estrategias. Esto solo afecta a lo que ves
Rango de los ejes	por defecto recorta el 1% de cada extremo, porque cuatro estrategias absurdas te aplastan el gráfico. `completo` enseña todo
Otra muestra	otros 100 puntos al azar, por si te ha tocado una muestra rara

Filtrar

Aquí es donde el panel deja de describir y empieza a servir para decidir. `+ añadir condición` añade una regla: por ejemplo *quédate solo con el 20% de mejor Sharpe en el IS*. Las opciones `% mejores` y `% peores` cortan por percentil; `>`, `≥`, `<` y `≤` cortan por un valor que escribes tú.



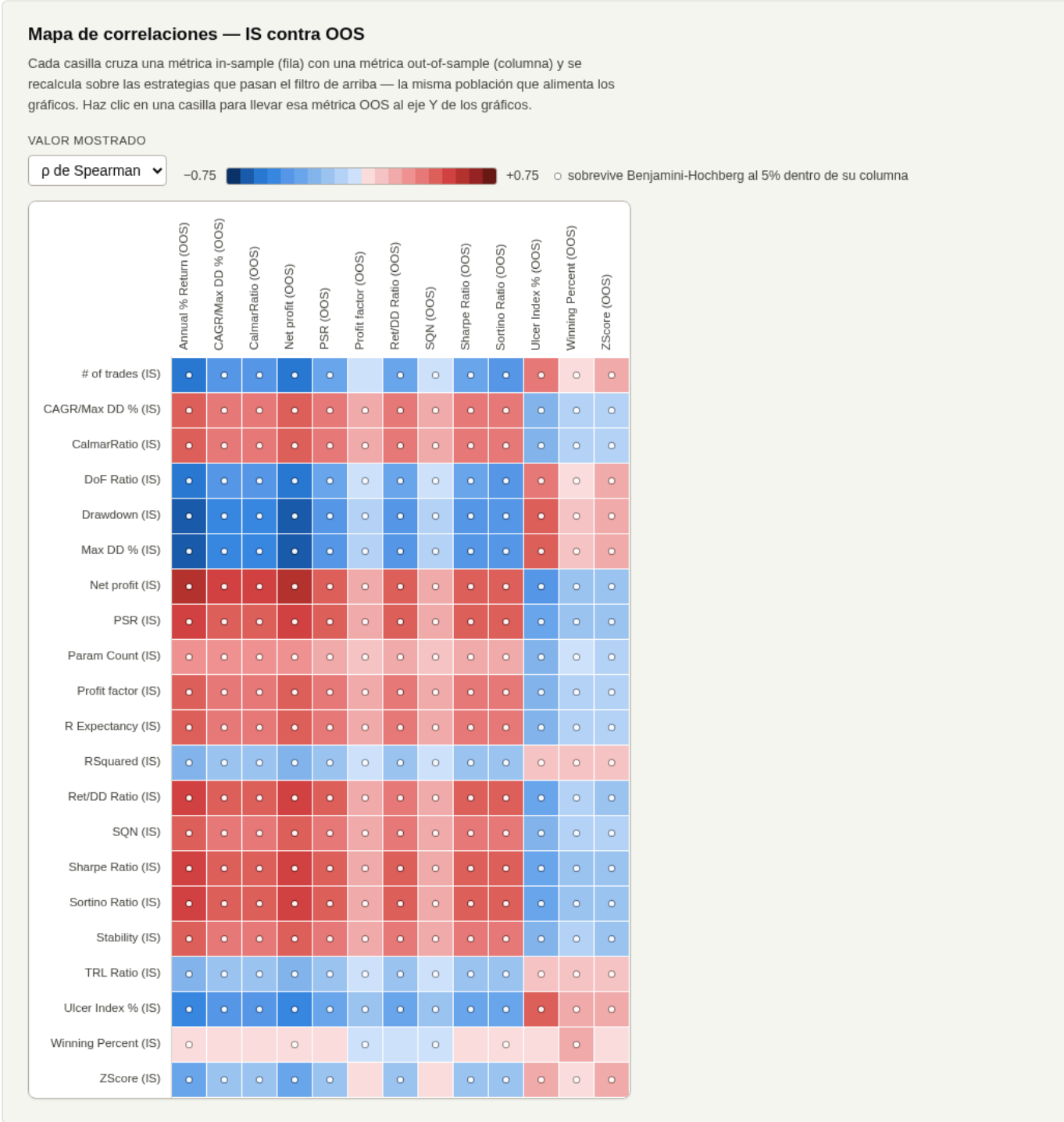
- **Gris** = las 10.000 estrategias. **Azul** = las que pasan el filtro.
- Arriba te dice cuántas sobreviven y **cómo cambia el resultado OOS típico** frente a no filtrar.
- Todos los números se recalculan sobre las que sobreviven.

Cada condición se evalúa contra la población **completa**. "Top 20%" es el 20% de las 10.000, no el 20% de lo que quedaba de una condición anterior. Varias condiciones se acumulan (tienen que cumplirse todas).

En el ejemplo de arriba: filtrando por el 20% de mejor Sharpe IS sobreviven 2.034 estrategias y la mediana del Profit factor OOS sube de 0,950 a 0,990.

El mapa de correlaciones

Debajo de los gráficos hay una tabla de colores que cruza **todas** las métricas del IS (filas) contra todas las del OOS (columnas). Es la vista de pájaro: dónde hay relación y dónde no, sin ir métrica por métrica.



- **Rojo** = relación positiva (más de esa métrica IS va con más resultado OOS). **Azul** = negativa. **Casi blanco** = nada.
- La escala tiene **20 escalones**, y sus extremos son los que marca la leyenda — no −1 y +1. A escala completa todo saldría del mismo color, porque estas correlaciones viven dentro de ±0,3.

- **El puntito blanco** en una casilla significa que ese cruce aguanta la corrección Benjamini- Hochberg dentro de su columna. Con 10.000 estrategias sobrevive casi todo; en cuanto pones un filtro y bajan los supervivientes, los puntos empiezan a desaparecer. Ahí es donde la marca dice algo.
- **Pasa el ratón** por una casilla para ver ρ, r y p exactos; **haz clic** y esa métrica OOS se va al eje Y de los gráficos de arriba.
- Se recalcula con el filtro puesto, igual que todo lo demás.

La tabla de estrategias

Más abajo están las cifras exactas, una fila por estrategia de las que pasan el filtro.

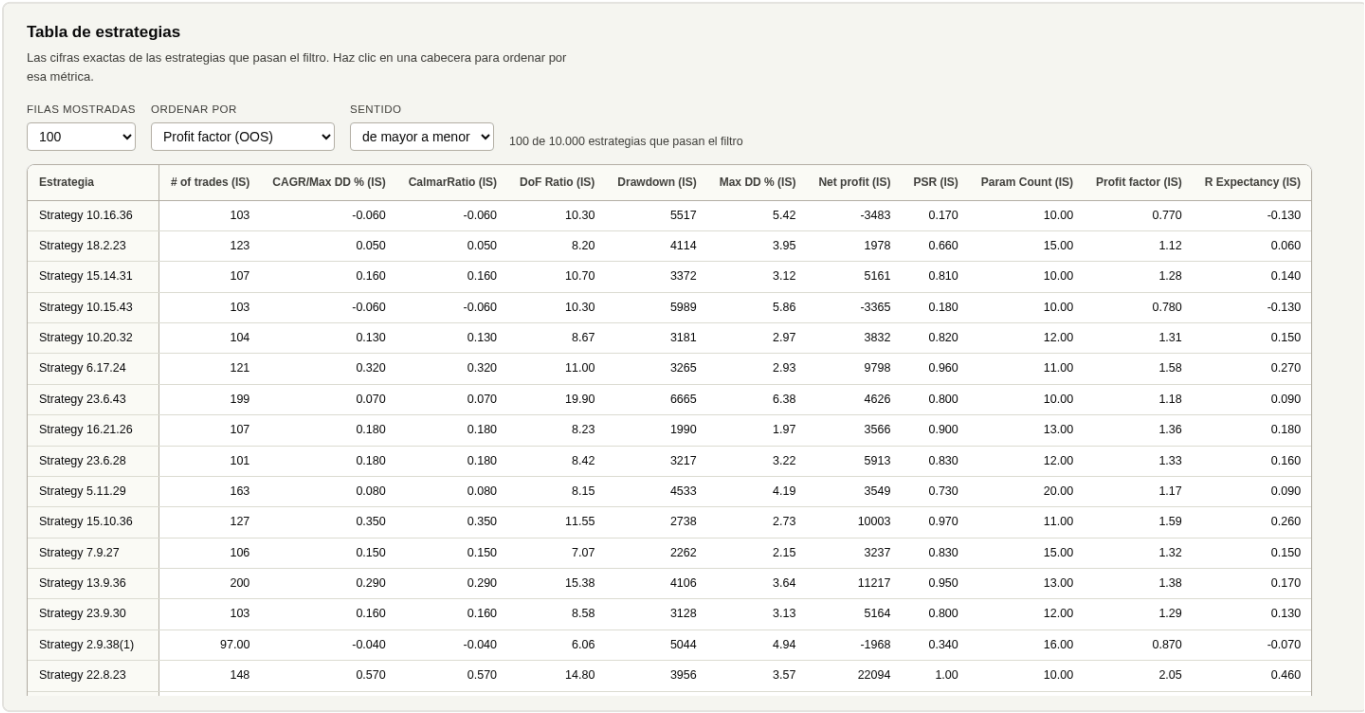


Tabla de estrategias

Las cifras exactas de las estrategias que pasan el filtro. Haz clic en una cabecera para ordenar por esa métrica.

FILAS MOSTRADAS: 100 | ORDENAR POR: Profit factor (OOS) | SENTIDO: de mayor a menor | 100 de 10.000 estrategias que pasan el filtro

Estrategia	# of trades (IS)	CAGR/Max DD % (IS)	CalmarRatio (IS)	DoF Ratio (IS)	Drawdown (IS)	Max DD % (IS)	Net profit (IS)	PSR (IS)	Param Count (IS)	Profit factor (IS)	R Expectancy (IS)
Strategy 10.16.36	103	-0.060	-0.060	10.30	5517	5.42	-3483	0.170	10.00	0.770	-0.130
Strategy 18.2.23	123	0.050	0.050	8.20	4114	3.95	1978	0.660	15.00	1.12	0.060
Strategy 15.14.31	107	0.160	0.160	10.70	3372	3.12	5161	0.810	10.00	1.28	0.140
Strategy 10.15.43	103	-0.060	-0.060	10.30	5989	5.86	-3365	0.180	10.00	0.780	-0.130
Strategy 10.20.32	104	0.130	0.130	8.67	3181	2.97	3832	0.820	12.00	1.31	0.150
Strategy 6.17.24	121	0.320	0.320	11.00	3265	2.93	9798	0.960	11.00	1.58	0.270
Strategy 23.6.43	199	0.070	0.070	19.90	6665	6.38	4626	0.800	10.00	1.18	0.090
Strategy 16.21.26	107	0.180	0.180	8.23	1990	1.97	3566	0.900	13.00	1.36	0.180
Strategy 23.6.28	101	0.180	0.180	8.42	3217	3.22	5913	0.830	12.00	1.33	0.160
Strategy 5.11.29	163	0.080	0.080	8.15	4533	4.19	3549	0.730	20.00	1.17	0.090
Strategy 15.10.36	127	0.350	0.350	11.55	2738	2.73	10003	0.970	11.00	1.59	0.260
Strategy 7.9.27	106	0.150	0.150	7.07	2262	2.15	3237	0.830	15.00	1.32	0.150
Strategy 13.9.36	200	0.290	0.290	15.38	4106	3.64	11217	0.950	13.00	1.38	0.170
Strategy 23.9.30	103	0.160	0.160	8.58	3128	3.13	5164	0.800	12.00	1.29	0.130
Strategy 2.9.38(1)	97.00	-0.040	-0.040	6.06	5044	4.94	-1968	0.340	16.00	0.870	-0.070
Strategy 22.8.23	148	0.570	0.570	14.80	3956	3.57	22094	1.00	10.00	2.05	0.460

Ordena haciendo clic en cualquier cabecera, o con los desplegables. `filas mostradas` limita cuántas se pintan (100 por defecto) — no cambia el filtro, solo lo que ves. Sirve para lo que un gráfico no sirve: ver quién es exactamente la estrategia que está en esa esquina de la nube.

Al final de la página

Hay cuatro fichas explicando **r de Pearson**, **ρ de Spearman**, **el p-valor** y **Benjamini-Hochberg**: qué mide cada uno, y sobre todo qué no. Están ahí porque el panel se lo acabas enseñando a alguien, y esas tres cifras no se explican solas.

Cómo se lee `isOos.md`

Es lo mismo en texto, pero calculado sobre las 10.000 y sin poder filtrar. Tiene tres partes:

Persistencia — si cada métrica mantiene su valor del IS al OOS. La columna `OOS/IS` por debajo de 1 significa que empeora al salir del histórico, que es lo normal.

Predicción — para cada resultado OOS, el ranking de métricas del IS que lo anticipan.

La columna `BH` — un ✓ significa que ese resultado aguanta la corrección por haber probado muchas métricas a la vez. Sin ✓, ignóralo aunque el número se vea bien: probando 21 métricas, una sale bien por azar.

```
| IS metric          |      ρ |      r |         p | BH |
| Sharpe Ratio (IS)  | +0.223 | +0.175 | 2.04e-113 | ✓  |
| PSR (IS)           | +0.220 | +0.175 | 3.14e-110 | ✓  |
| # of trades (IS)   | -0.019 | -0.052 |    0.0519 |    |
```

Un ejemplo completo

Pregunta: **¿me sirve el Sharpe del histórico para elegir estrategias de oro?**

```
cd ~/Desktop/AlgoProject
python3 -m studies.screening.isOos.report --project XAUUSD --databank OOS
```

```
10000 strategies, 21 IS metrics x 13 OOS
  /home/sergioguslw/Desktop/AlgoData/reports/XAUUSD/OOS/2026-09-04/isOos/isOos.html
  /home/sergioguslw/Desktop/AlgoData/reports/XAUUSD/OOS/2026-09-04/isOos/explorer.html
```

Abres el panel, dejas `Profit factor (OOS)` en el eje Y y miras el primer gráfico: `Sharpe Ratio (IS)`, ρ +0.223. Hay señal, pero floja.

Añades el filtro *Sharpe Ratio (IS) — top % — 20*. Arriba aparece:

> **2034** de 10.000 pasan el filtro (**20.3%**) · mediana de Profit factor (OOS): **0.990** filtrado vs 0.950 sin filtrar

Respuesta: **sí, pero poco.** Descartando el 80% de las estrategias, la mediana del profit factor OOS pasa de 0,950 a 0,990 — sigue por debajo de 1. Sirve como un filtro más, no como *el* filtro.

Y algo que solo se ve haciéndolo: **con el filtro puesto, el ρ del propio Sharpe se desploma de +0.223 a +0.088**, y pasa a mandar otra métrica. Dentro del 20% bueno, el Sharpe ya no distingue. Ese es el motivo de que el panel recalcule todo al filtrar en vez de enseñarte siempre los mismos números.

IS contra OOS, operación por operación

La parte de población compara **métricas de estrategias enteras**. Esta compara, dentro de **una** estrategia, **cómo son sus operaciones** dentro y fuera de muestra. Si fuera de muestra las operaciones ganan menos, duran distinto o sufren más antes de cerrarse, aquí se ve; y se ve como un número, no como una impresión.

```
python3 -m studies.screening.isOos.report --project Test_USDJPY_donchianUpperCrossUp_M30 --
databank Results
```

```
200 estrategias IS contra OOS por operación (0 sin una de las dos muestras) ->
.../reports/Test_USDJPY_donchianUpperCrossUp_M30/Results/2026-09-27/isOos/estrategias
```

(~7 segundos para 200 estrategias y 302.000 operaciones; unos 23 KB por página.)

Test_USDJPY_donchianUpperCrossUp_M30 / Results — dentro de muestra contra fuera — Strategy 1.15.54

IS contra OOS, operación por operación

IS contra OOS, operación por operación

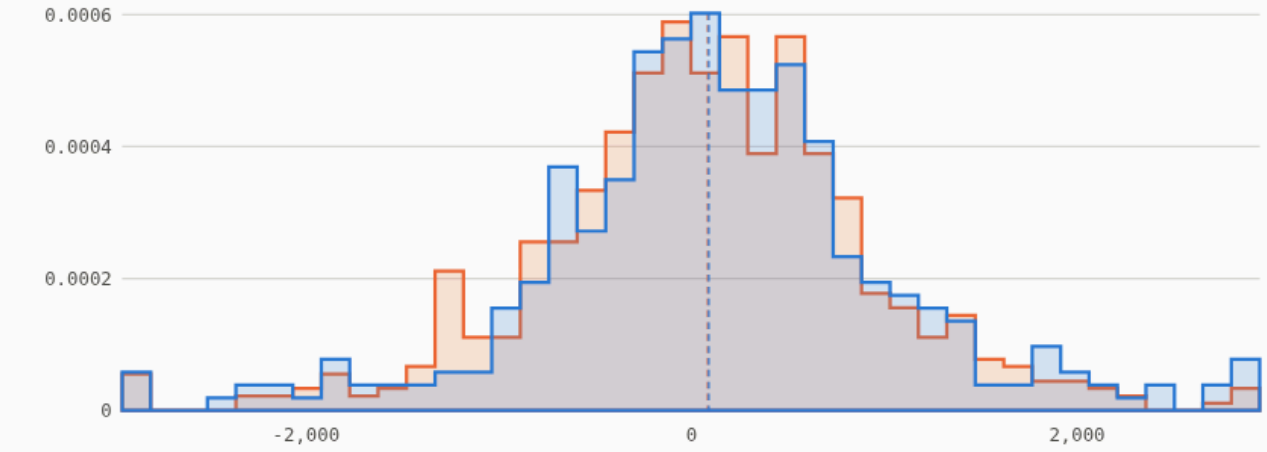
Las mismas cuatro medidas por operación dentro y fuera de muestra. Sólo medidas por operación: el beneficio neto y el drawdown crecen con la longitud de la ventana y no se comparan así. El múltiplo R espera a que la estrategia lleve stop (paso 24).

Aquí se ve la combinación por defecto de Unidad del dinero (USD). Las demás están en el resultado JSON y en la ventana.

unidad: USD

Retorno por operación

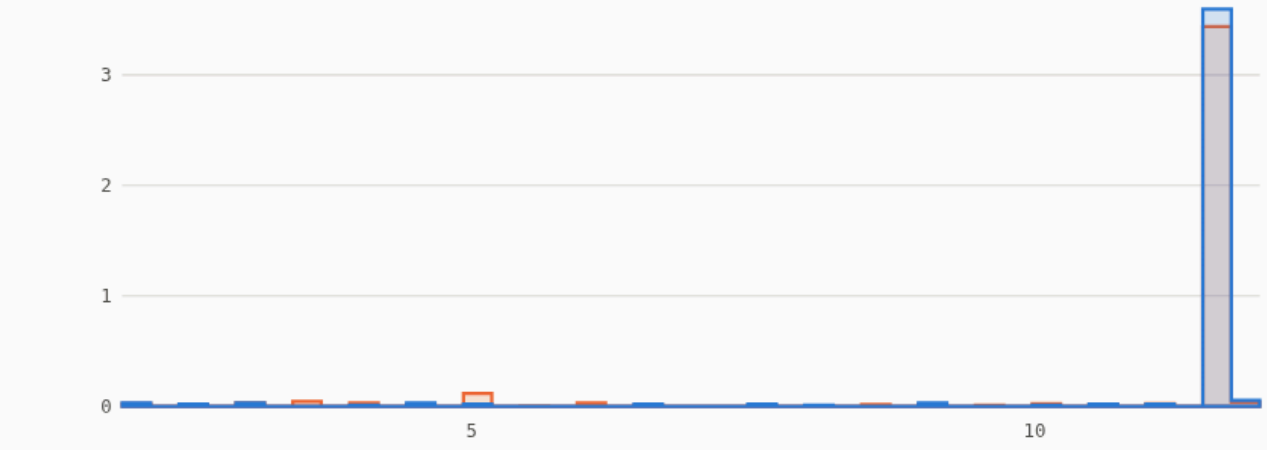
Cada curva es una densidad —el área bajo cada una vale 1—, así IS y OOS se comparan aunque tengan distinto número de operaciones. Las colas más allá del eje se suman a la primera y la última barra.



IS · n 610 · mediana 83.47 OOS · n 349 · mediana 90.22 desplazamiento de la mediana 6.75 · KS p = 0.9102

Duración de cada operación

Cada curva es una densidad —el área bajo cada una vale 1—, así IS y OOS se comparan aunque tengan distinto número de operaciones. Las colas más allá del eje se suman a la primera y la última barra.



IS · n 610 · mediana 11.50 OOS · n 349 · mediana 11.50 desplazamiento de la mediana 0 · KS p = 0.7629

unidad: USD



Tres gráficos por estrategia, cada uno con las dos muestras **superpuestas**: IS en naranja, OOS en azul, cada una con su mediana en línea discontinua de su color. (La distribución de duración de cada operación salió el 2026-09-29, a petición del dueño.)

gráfico	qué es cada operación
Retorno por operación	lo que ganó o perdió al cerrarse
MAE	lo peor que llegó a ir mientras estaba abierta (negativo)
MFE	lo mejor que llegó a ir mientras estaba abierta

- **Son densidades, no conteos.** El área bajo cada curva vale 1. Por eso se pueden poner una encima de otra aunque el OOS tenga la mitad de operaciones: lo que compara es la **forma**.
- **Debajo de cada gráfico**: `n` y la mediana de cada muestra, el **desplazamiento de la mediana** (OOS menos IS, en la unidad del gráfico) y el **p de Kolmogorov-Smirnov**. Un p bajo (por debajo de 0,05) dice que las dos muestras no parecen venir de la misma distribución; uno alto, que no hay prueba de que hayan cambiado.
- **Las colas**: el eje va del percentil 0,5 al 99,5 de las dos muestras juntas; las operaciones más extremas se suman a la primera y la última barra (`trades.tail_pct`).
- **Unidad del dinero** (retorno, MAE, MFE): el resultado trae tres lecturas y la ventana tiene un desplegable. `USD` es la cifra tal cual la exporta SQX; `% del saldo` la divide por el saldo antes de la operación; `USD por lote` la divide por el tamaño. Importa porque el tamaño cambia entre muestras (en el ejemplo, de 3,3 lotes de mediana en IS a 4,6 en OOS): en USD eso parece un cambio de la estrategia sin serlo. La página HTML y el `.md` enseñan `USD` .

Qué no está, a propósito: el beneficio neto y el drawdown (crecen con la longitud de la ventana y no se comparan así) y el **múltiplo R**, que espera a que la estrategia lleve stop (que ya no existe como paso). Tampoco hay veredicto: describe, no filtra, y no escribe `verdict.csv` .

Qué NO te dice

- **No dice que una estrategia sea buena.** Habla de tendencias en miles de estrategias. Una concreta puede estar en cualquier sitio de esa nube.
- **No dice que la métrica cause el resultado.** Dice que van juntas.
- **Solo vale para este activo y este periodo.** Que el Sharpe funcione en XAUUSD no dice nada de los índices. Hay que repetirlo por activo.
- **No te dice si el filtro merece la pena.** Ves que la mediana sube y cuántas estrategias pierdes, pero no si esa subida aguanta o es ruido de haber probado muchos filtros. Para eso está la página 2, *Barrido de filtros*, que prueba 210 candidatos con intervalo de confianza y corrección.
- **No hay costes reales metidos.** Son los números que da SQX con su configuración de comisiones.

Si algo falla

`FileNotFoundError: .../metrics/XAUUSD/OOS/metrics.csv` No has hecho el paso 1 para ese databank, o escribiste otro nombre. Los nombres son los de SQX, con mayúsculas y todo.

El CSV sale con una sola línea El worker exportó antes de terminar de cargar las estrategias. Vuelve a lanzar el paso 1.

`explorer.html` tarda en abrir Normal: son unos megas porque lleva dentro las 10.000 estrategias, que es lo que le permite recalcular al filtrar sin volver a pedir nada.

El panel se ve en blanco Ábrelo con doble clic o con `xdg-open`. Arrastrarlo a una pestaña también funciona.

2. Barrido de filtros — cuánto te da de verdad filtrar, y a cambio de qué

Qué pregunta responde

El análisis IS/OOS (página 1) te dice **qué métricas del histórico llevan información** sobre el futuro. No te dice qué hacer con ellas. Esta página sí:

> *"Si me quedo solo con el 10% de mejor CAGR/Max DD del histórico, ¿cuánto mejora el resultado fuera de muestra, cuántas estrategias pierdo, y esa mejora es real o es que he probado muchas combinaciones hasta que una salió bien?"*

El comando prueba **210 filtros candidatos** de una tacada — cada métrica IS cortada por arriba y por abajo al 5, 10, 20, 30 y 50% — y de cada uno reporta la mejora, el intervalo de confianza de esa mejora y cuántas estrategias sobreviven.

Cuándo lo usas, y cuándo no

Lo usas para decidir dos cosas concretas: qué condiciones de aceptación poner en la generación de SQX, y con qué criterio cribar un databank antes de pasar al testeo serio.

No lo usas con estrategias que ya fueron seleccionadas por su resultado OOS — igual que la página 1, ahí mides tu propia selección. Tampoco lo uses para elegir *una* estrategia: habla de poblaciones.

Y no lo uses como coartada: que un filtro salga con ✓ no significa que dé dinero. Significa que la mejora que muestra no se explica por el azar del muestreo.

Antes de empezar

Lo mismo que la página 1, y nada más:

1. El CSV de métricas del databank ya exportado (`python3 -m sqx.export.export_metrics --project XAUUSD --databank OOS`).
2. Nada de SQX. Este comando no lo toca: puedes correrlo con la GUI abierta.

Cómo se ejecuta

```
cd ~/Desktop/AlgoProject
python3 -m studies.screening.filters.report --project XAUUSD --databank OOS
```

flag	obligatorio	qué hace
`--project`	sí	nombre del proyecto tal y como aparece en SQX
`--databank`	sí	nombre del databank, p. ej. `OOS`
`--target`	no	columna OOS a mejorar. Repetible. Por defecto `Sharpe Ratio (OOS)` y `Ret/DD Ratio (OOS)`
`--top`	no	cuántos filtros lista por resultado. Por defecto 15

Tarda unos 40 segundos con 10.000 estrategias y dos objetivos: casi todo es el bootstrap, que son 2.000 remuestreos por cada filtro candidato; el que no cruza el cero ni una vez se vuelve a sortear hasta 6.000 (con 210 filtros), para que un único filtro fuerte pueda nombrarse.

Qué produce

```
~/Desktop/AlgoData/reports/XAUUSD/OOS/2026-09-04/filters/
    filters.md       la tabla de filtros, en texto (antes `improvement.md`)
    filters.html     la misma tabla como página
    filters.json     el resultado entero, en el contrato de estudios
    manifest.json    de qué CSV salió y con qué versión del código
```

Va en su propia subcarpeta `filters/` dentro del reporte del día, para no pisar el `manifest.json` del análisis IS/OOS. Se sobrescribe si lo repites el mismo día; los días anteriores no se tocan.

Cómo se lee el resultado

```
## Sharpe Ratio (OOS)

Unfiltered: 10,000 strategies · median -0.280 · 25.6% above 0.
108 of the 210 filters judged both improve the median and survive the correction.

| filter                     |  kept | median | Δ median |    95% CI on Δ  | hit % | Δ hit pp |
p | BH |
| CAGR/Max DD % (IS) top 5%  |   511 | -0.020 |   +0.260 | [+0.230,+0.300] |  47.7 |    +22.2 |
5.00e-04 | ✓  |
| Sharpe Ratio (IS) top 5%   |   528 | -0.020 |   +0.260 | [+0.230,+0.300] |  47.9 |    +22.4 |
5.00e-04 | ✓  |
| CAGR/Max DD % (IS) top 10% | 1,018 | -0.030 |   +0.250 | [+0.230,+0.280] |  46.3 |    +20.7 |
5.00e-04 | ✓  |
| PSR (IS) top 10%           | 1,045 | -0.030 |   +0.250 | [+0.220,+0.270] |  45.7 |    +20.2 |
5.00e-04 | ✓  |
| Profit factor (IS) top 20% | 2,019 | -0.050 |   +0.230 | [+0.200,+0.240] |  43.7 |    +18.1 |
5.00e-04 | ✓  |
```

Columna por columna:

columna	qué es
filter	la condición. `top 10%` = quedarse con el 10% de valor más alto de esa métrica IS; `bottom` = el más bajo
kept	cuántas estrategias sobreviven. **Este es el precio del filtro**
median	la mediana del resultado OOS entre las supervivientes
Δ median	cuánto sube esa mediana frente a no filtrar. Es la ganancia
95% CI on Δ	el intervalo de esa ganancia. **Si incluye el 0, la mejora no se distingue del ruido** — es la columna que decide
hit %	qué porcentaje de las supervivientes acaba por encima del punto de equilibrio (0 para Sharpe y Ret/DD; 1 para profit factor)
Δ hit pp	lo mismo en puntos porcentuales frente a no filtrar. Suele ser la cifra más intuitiva
p	p-valor bootstrap. **Tiene suelo en 1/6000 = 1.67e-04** con 210 filtros (por debajo del primer peldaño de Benjamini-Hochberg, 0,05/210): cuando está pegado ahí, mira el intervalo, no el p
BH	✓ = aguanta la corrección por haber probado 210 filtros a la vez

Lo que hay que mirar primero es `kept` y `Δ hit pp`, en ese orden. Un filtro que sube 20 puntos la tasa de acierto tirando el 90% de las estrategias es una decisión distinta a uno que sube 18 tirando el 80%, y la elección depende de cuántas estrategias necesites al final del embudo.

Un filtro que deja **menos de 200 supervivientes ni se evalúa**. Con 12 estrategias vivas cualquier mejora es un espejismo, y la regla está fijada de antemano para que no se decida caso por caso.

Un ejemplo completo

Pregunta: **¿qué condición de aceptación pongo en la generación de XAUUSD?**

```
cd ~/Desktop/AlgoProject
python3 -m studies.screening.filters.report --project XAUUSD --databank OOS
```

```
210 candidate filters over 10000 strategies, 2 outcomes
  /home/sergioguslw/Desktop/AlgoData/reports/XAUUSD/OOS/2026-09-04/filters/filters.md
```

En `filters.md`, para `Ret/DD Ratio (OOS)`:

> Best surviving filter: **Sharpe Ratio (IS) top 5%** — keeps 528 strategies, median 0.015 (+0.485), hit rate 50.4% (+23.7 pp).

Lectura en cristiano: sin filtrar, **el 26,7%** de las estrategias acaba con Ret/DD positivo fuera de muestra. Quedándote con el 5% de mejor Sharpe del histórico, **el 50,4%** — casi el doble de tasa de acierto, a cambio de descartar 19 de cada 20 estrategias generadas.

Dónde deja de pagar apretar más. Esa es la pregunta que contesta tener los cortes 5 y 10 juntos:

corte	sobreviven	acierto OOS
sin filtro	10.000	26,7%
top 10%	~1.020	48,6%
top 5%	~510	50,4%

El primer corte compra **22 puntos**; apretar del 10% al 5% compra **menos de 2 más** y te deja la mitad de estrategias. La curva se aplana: **el 10% es el punto razonable** salvo que necesites lo mejor de lo mejor y te sobren candidatas.

Una segunda cosa que se ve sola en la tabla: `CAGR/Max DD %`, `CalmarRatio`, `R Expectancy`, `SQN`, `Profit factor`, `Sharpe` y `Sortino` al 10% dan **prácticamente el mismo resultado**. No son siete filtros: son siete formas de medir lo mismo. Aplicarlos a la vez no multiplica nada, solo estrecha el embudo.

Qué NO te dice

- **No dice que combinando filtros mejore más.** Solo se barren condiciones sueltas. Dos condiciones a la vez son otro espacio de búsqueda y necesitan otra corrección.
- **No dice que el filtro valga para otro activo.** Todo esto es XAUUSD y este periodo. Hay que repetirlo por activo antes de generalizar nada.
- **No dice nada sobre costes reales.** Son los números de SQX con su configuración de comisiones.
- **No convierte una mejora estadística en dinero.** Sube la proporción de estrategias decentes en lo que sale del generador; lo que hagas después con ellas es otro problema.
- **El intervalo no cubre el riesgo de que el periodo OOS sea atípico.** Cubre el muestreo dentro de estos datos, no que 2023 se parezca a 2026.

Si algo falla

lo que ves	qué pasa
`FileNotFoundError: .../metrics.csv`	no has exportado ese databank todavía. Corre `export_metrics` primero
`KeyError: 'Sharpe Ratio (OOS)'`	la vista de SQX no trae esa columna en OOS. Pásale `--target` con una que sí exista
tarda varios minutos	normal si el databank es mucho mayor de 10.000 estrategias: el bootstrap crece con el número de supervivientes

3. Comparar muestras — ¿se cumplen las conclusiones en otra generación?

Qué pregunta responde

Las páginas 1 y 2 sacan conclusiones de **una** población: *"filtrando por el 10% de mejor Sharpe IS, la tasa de acierto OOS sube del 27% al 48%"*. Eso es una **predicción**, y una predicción se comprueba generando estrategias nuevas y viendo si se cumple.

Esta página compara varias generaciones del mismo activo — la aleatoria de referencia y las que hiciste después con distintas condiciones o distintos periodos — y responde a dos cosas:

1. **¿Entregó el filtro lo que prometía?** El umbral que salió de la muestra aleatoria se aplica a la muestra nueva y se compara lo predicho con lo observado, con su intervalo.
2. **¿Siguen mandando las mismas métricas?** Si la muestra nueva ordena los predictores igual que la de referencia, la conclusión aguanta; si los reordena, no era estable.

Cuándo lo usas, y cuándo no

Lo usas cuando tienes **generaciones independientes**: databanks con estrategias nuevas, creadas en corridas distintas de SQX, o la misma población medida en otro periodo.

No lo usas para comparar una población con un subconjunto suyo. Si `OOS-sharpe` no son estrategias nuevas sino las 10.000 de siempre filtradas, no hay nada que replicar: sabes de antemano lo que va a salir, y el filtro del panel (página 1) ya te lo enseña. La comparación solo significa algo entre muestras que no comparten estrategias.

⚠ **Y no compares correlaciones entre muestras seleccionadas de forma distinta.** Si generaste `OOS-sharpe` exigiendo Sharpe alto, dentro de esa muestra el Sharpe apenas varía, así que su correlación con cualquier resultado se desploma **por construcción**. Eso no significa que la conclusión fallara. El informe lo detecta y lo avisa, y por eso la comparación que manda es la del **resultado**, no la de la correlación.

Antes de empezar

Cada databank que quieras comparar necesita su exportación de métricas hecha (se lee de los `.sqx` en medio segundo, y la ventana la hace sola al elegir el databank):

```
python3 -m sqx.export.export_metrics --project XAUUSD --databank OOS-sharpe
```

El comando usa solo las métricas comunes a todas las muestras y te dice cuántas son.

Cómo se ejecuta

```
cd ~/Desktop/AlgoProject
python3 -m studies.screening.replication.report --project XAUUSD --reference OOS \
    --databank OOS-sharpe --databank OOS-rExpectancy --databank OOS-hasta2026
```

flag	obligatorio	qué hace
`--project`	sí	nombre del proyecto tal y como aparece en SQX
`--reference`	sí	el databank del que salieron las conclusiones. Normalmente la generación **aleatoria**
`--databank`	sí	databank a comprobar. **Repetible**: uno por cada muestra
`--target`	no	columna OOS a comparar. Repetible. Por defecto `Sharpe Ratio (OOS)` y `Ret/DD Ratio (OOS)`

Tarda unos 25 segundos con tres muestras y dos objetivos. No toca SQX.

Qué produce

```
~/Desktop/AlgoData/reports/XAUUSD/_comparison/2026-09-04/replication/
    replication.md   la comparación (antes `comparison.md`, suelto en la carpeta del día)
    replication.html la misma comparación como página
    replication.json el resultado entero, en el contrato de estudios
    manifest.json    qué muestras, de qué fecha de exportación, con qué código
```

Va en `_comparison/` porque no pertenece a ningún databank: habla de varios a la vez. El guion bajo lo mantiene separado de las carpetas que sí son databanks.

Cómo se lee el resultado

Primera tabla: dónde acabó cada muestra

```
| databank         | strategies | median | hit % | Δ hit pp |  95% CI on Δ   |
| OOS              |     10,000 | -0.280 |  25.6 |        — |              — |
| _selftest_half   |      5,000 | -0.280 |  25.7 |     +0.1 | [-1.4, +1.6]   |
| _selftest_sharpe |      1,043 | -0.030 |  45.8 |    +20.3 | [+17.0, +23.3] |
```

`Δ hit pp` es la diferencia en tasa de acierto contra la referencia, y **el intervalo es lo que decide**: si incluye el 0, esa muestra no se distingue de la de referencia. Arriba, la primera no se distingue (era media población al azar) y la segunda sí (era el 10% de mejor Sharpe).

Segunda tabla: ¿entregó el filtro lo prometido?

```
| filter                   | predicted hit % | % of sample passing |   n | observed hit % |
Δ |  95% CI on Δ  |
| Sharpe Ratio (IS) top 5%  |            47.9 |                50.6 | 528 |           47.9 |
+0.0 | [-6.1, +6.1]  |
| CAGR/Max DD % (IS) top 5% |            47.7 |                47.8 | 499 |           47.3 |
-0.5 | [-6.9, +5.5]  |
```

columna	qué es
predicted hit %	lo que ese filtro consiguió **en la muestra de referencia**. Es la promesa
% of sample passing	cuánto de la muestra nueva pasa ese mismo umbral. **Cerca de 100% significa que la muestra se generó para cumplirlo**
observed hit %	lo que consiguen de verdad, en la muestra nueva, las que pasan el umbral
Δ y su intervalo	observado menos prometido. Si el intervalo incluye el 0, **el filtro cumplió**: no se desvía de lo que prometía

⚠ **Seis filas de acuerdo no son seis confirmaciones.** Los filtros de esta lista se solapan casi del todo (seleccionan prácticamente las mismas estrategias), así que se mueven juntos. Una muestra con suerte sube las seis a la vez. Mira el intervalo, no cuántas filas apuntan al mismo lado.

Tercera parte: ¿mandan las mismas métricas?

```
**_selftest_half** ranks the predictors like the reference at ρ +0.995.

**_selftest_sharpe** ranks the predictors like the reference at ρ +0.407.
Selected on 12 metric(s) — # of trades (IS), DoF Ratio (IS), Drawdown (IS), Max DD % (IS) —
whose own correlations are attenuated here by construction.
```

Un ρ cercano a +1 significa que las dos muestras están de acuerdo en qué métricas predicen. Cuando baja, **mira primero la línea de debajo**: si la muestra fue seleccionada sobre esas métricas, la caída es aritmética y no dice nada. Si no lo fue, entonces sí: la conclusión no era estable.

Un ejemplo completo

Las salidas de arriba son de una **prueba del propio comando**, hecha el 2026-09-04 partiendo la exportación de XAUUSD/OOS en dos muestras artificiales para comprobar que detecta lo que debe:

- `_selftest_half` — media población al azar. **Debe salir "no hay diferencia"**, y sale: +0.1 pp con intervalo [−1.4, +1.6].
- `_selftest_sharpe` — el 10% de mejor Sharpe IS. **Debe replicar la predicción**, y sale: predicho 47.9%, observado 47.9%, Δ +0.0.

Con tus muestras de verdad el comando es el mismo, cambiando los nombres de los databanks. Las dos muestras de prueba se borraron después: no están en el data root.

Qué NO te dice

- **No dice por qué una muestra falló.** Dice que lo hizo. La causa —periodo distinto, plantilla distinta, otra configuración del generador— la sabes tú, y el informe no puede adivinarla.
- **No corrige por cuántas muestras compares.** Comparar diez muestras y quedarte con la que mejor salió es el mismo problema de siempre. Decide de antemano qué esperas de cada una.
- **No compara periodos distintos como si fueran el mismo experimento.** Una muestra "hasta 2026" difiere de la referencia en el periodo *y* en las estrategias; si sale distinta, no sabrás cuál de las dos cosas lo causó a menos que dejes una fija.
- **No mide dinero.** Tasa de acierto y mediana, con los números y las comisiones de SQX.

Si algo falla

lo que ves	qué pasa
`FileNotFoundError: .../metrics.csv`	ese databank no está exportado todavía
`0 in-sample metrics common to every sample`	alguna muestra es una exportación antigua hecha con otra vista. Vuelve a exportarla: ahora todas salen con las mismas columnas
una muestra con muy pocas estrategias	los intervalos se abrirán tanto que no dirán nada. Por debajo de ~200 no merece la pena compararla

4. Decaimiento — cuánto del edge sobrevive fuera de muestra, y si te la quedas o no

Qué pregunta responde

Generas estrategias sobre el histórico y luego las pruebas en un tramo que no vieron. Casi todas empeoran. Este análisis te dice, **estrategia por estrategia**, cuánto empeoró y si lo que le queda es un edge de verdad o el rastro de una racha con suerte. Termina con una sentencia por estrategia: `MANTENER`, `DUDOSA` o `DESCARTAR`.

La pregunta que hay detrás es la de Marcos López de Prado: una caída fuerte de rendimiento al salir de la muestra es el síntoma clásico del sobreajuste. Pero la caída sola engaña, y por eso aquí no se decide con ella. Se decide con tres cosas a la vez:

1. **¿El Sharpe fuera de muestra se distingue de cero?** Un Sharpe de 0,4 medido sobre cinco años tiene un error de ±0,45. Es decir: podría ser cero. La columna `t` es el Sharpe dividido por su propio error; por debajo de 1,65 no puedes afirmar que haya nada. Ojo: ese 1,65 **no** es un «5 % a una cola» exacto, porque la curva diaria de SQX apunta el punto más bajo de cada día y la `t` sale entre un 12 y un 22 % más pequeña de lo que sería de cierre a cierre. Para juzgar si hay edge, la cifra buena es `drift_excess_t` de la puerta OOS, que sale de las operaciones.
2. **¿Ganó todos los años, o solo alguno?** Con cinco años por delante, ganar tres es lo que sale tirando una moneda.
3. **¿De dónde sale el beneficio?** Si más de la mitad viene de un solo trimestre, no tienes una estrategia: tienes un billete de lotería premiado. La columna `concentración` por encima del 100% significa que fuera de ese trimestre la estrategia **pierde**.

Cuándo lo usas, y cuándo no

Úsalo cuando ya tienes un databank con estrategias que llevan resultado guardado en dos tramos — un IS y un OOS — y quieres saber con cuáles quedarte.

Cuándo NO sirve:

- Si el databank no tiene tramo fuera de muestra, no hay nada que comparar.
- Si ya filtraste ese mismo tramo OOS por rendimiento (por ejemplo "net profit > 0" sobre todo el OOS), el veredicto es **descriptivo, no predictivo**. Te dice lo que pasó, no lo que pasará: las estrategias que están ahí lo están precisamente porque ese tramo les salió bien. Para que fuera predictivo, el tramo con el que juzgas tendría que no haberse usado nunca para filtrar.
- No sirve para comparar plantillas entre sí sobre el papel. Agrupa por plantilla solo para que veas de dónde salen las supervivientes; no promedia nada, porque estrategias de plantillas distintas no son la misma población.

Antes de empezar

- El databank tiene que estar **volcado a disco**. SQX guarda las estrategias en memoria y solo escribe los `.sqx` al sincronizar; si la carpeta está vacía, abre SQX, entra en el databank y deja que sincronice.

- **No hace falta arrancar nada.** Este comando lee los `.sqx` directamente y no usa SQX ni el worker, así que puedes lanzarlo con la GUI abierta sin ningún riesgo.

Cómo se ejecuta

```
python3 -m studies.screening.decay.report --project XAUUSD --databank OOS \
        --split 2018-01-01 --end 2022-12-31
```

flag	obligatorio	qué hace
`--project`	sí	nombre del proyecto tal y como aparece en SQX
`--databank`	sí	nombre del databank, con sus espacios si los tiene
`--split`	sí	primer día del tramo fuera de muestra. Todo lo anterior es IS
`--end`	sí	último día que se mira. Lo posterior se ignora
`--role`	no	el worker que tiene el proyecto (`custodian` , `conductor`); sin él, el máster
`--is-databank`	no	el build que ese databank reteseó (`Results`): de ahí sale la parte IS, emparejada por nombre
`--strategy`	no	una sola estrategia, con el nombre que le da el databank (`"Strategy 1.26.46"`): escribe solo `estrategias/<nombre>.json` y su página, con los mismos números y veredicto que su fila del `verdict.csv` . **No toca** `decay.json` , `verdict.csv` ni el manifiesto, así que el resto del databank sigue como estaba. Es lo que hace «Run solo esta estrategia» en la ventana

En un proyecto de workflow, la IS y la OOS están en dos databanks (`Results` es el build, solo IS; `OOS` es el retest, solo OOS). Leyendo solo `OOS` , todas las estrategias salían sin Sharpe IS y sin retención (medido el 30-09: 0 de 296). Con `--is-databank Results` se pega la curva del build antes de `--split` con la del retest después. **La ventana lo pasa sola** cuando corres el estudio sobre `OOS` desde el paso 8:

```
python3 -m studies.screening.decay.report --project Test_USDJPY_donchianUpperCrossUp_H1 \
        --databank OOS --split 2018-01-01 --end 2022-12-31 --role custodian --is-databank
Results
```

Salida real (30-09, 60 estrategias tras el corte del paso 8): `Strategy 1.26.46` Sharpe IS 0,435, OOS 0,686, t 1,41 → DUDOSA; en total 9 DUDOSA y 51 DESCARTAR.

Tarda unos segundos para 834 estrategias. No toca SQX.

```
$ python3 -m tasks.reports.decay --project XAUUSD --databank OOS \
      --split 2018-01-01 --end 2022-12-31

834 estrategias → /home/sergioguslw/Desktop/AlgoData/reports/XAUUSD/OOS/2026-09-06
verdict
DUDOSA       522
DESCARTAR    308
MANTENER       4
```

Qué produce

En `~/Desktop/AlgoData/reports/<proyecto>/<databank>/<fecha>/decay/` (hasta el 25-09, sueltos en la carpeta del día):

archivo	qué es
`verdict.csv`	una fila por estrategia con todas las columnas, su identidad y su veredicto (antes `decay.csv`)
`decay.html` / `decay.json`	el mismo resumen como página y como resultado del contrato de estudios
`decay.md`	el resumen escrito: recuento, retención, desglose por plantilla y las que pasan todo
`manifest.json`	qué se analizó, con qué fechas y con qué versión del código

Los informes se acumulan por fecha: uno nuevo no borra el anterior.

Cómo se lee el resultado

Decaimiento — XAUUSD / OOS

834 estrategias · IS hasta 2018-01-01 · OOS 2018-01-01 → 2022-12-31 · informe 2026-09-06

Veredicto

veredicto	n
DUDOSA	522
DESCARTAR	308
MANTENER	4

Retención del Sharpe

Mediana 24% · p90 46% · máximo 110%

Ninguna estrategia con t ≥ 2 (el máximo es 1.94): el Sharpe fuera de muestra no se distingue de cero en ninguna.

Por plantilla

plantilla	n	mantener	descartar
Strategy 1	128	0	62
Strategy 2	169	0	69
Strategy 3	74	0	36
Strategy 4	105	3	29
Strategy 5	166	0	59
Strategy 6	192	1	53

Las que pasan todo

estrategia	Sharpe IS	Sharpe OOS	retiene	t	años+	concentr.
Strategy 4.16.45	1.46	1.08	74%	1.94	5	16%
Strategy 4.18.51	1.38	1.02	74%	1.87	5	18%
Strategy 4.17.47	1.38	1.02	74%	1.87	5	17%
Strategy 6.18.78	1.34	0.91	68%	1.74	5	17%

Las columnas del CSV, de izquierda a derecha:

columna	qué es	qué valor es bueno
`sharpe_is`	Sharpe anualizado en el tramo de entrenamiento	contexto, no criterio
`sharpe_oos`	Sharpe anualizado fuera de muestra	cuanto más alto, mejor
`retention`	`sharpe_oos / sharpe_is`: la fracción del edge que sobrevivió	100% sería no perder nada; en la práctica la mediana ronda el 25%
`t`	`sharpe_oos` dividido por su error típico	**la columna que manda.** Por debajo de 1,65 no hay evidencia de edge
`years_positive`	años del tramo OOS que cerró en verde	4 o 5 de 5
`worst_year`	el peor año, en dinero	mide lo que tendrías que aguantar
`concentration`	qué parte del beneficio salió de su mejor trimestre	por debajo del 40%. Por encima del 100%, pierde el resto del tiempo
`verdict`	la sentencia	ver abajo

Las tres sentencias:

- **`DESCARTAR`** — el Sharpe fuera de muestra es negativo, o perdió en la mitad de los años, o más del 60% del beneficio viene de un solo trimestre. Cualquiera de las tres basta.
- **`MANTENER`** — `t` ≥ 1,65, ganó en 4 de 5 años y ninguna concentración por encima del 40%.
- **`DUDOSA`** — todo lo demás. No es un aprobado flojo: es "los datos no dan para decidir". La mayoría cae aquí, y eso **es** el resultado, no un fallo del análisis.

Un ejemplo completo

Sobre el databank `OOS` de XAUUSD, 834 estrategias que ya habían pasado un filtro de net profit positivo fuera de muestra:

```
$ python3 -m studies.screening.decay.report --project XAUUSD --databank OOS \
        --split 2018-01-01 --end 2022-12-31

834 estrategias → /home/sergioguslw/Desktop/AlgoData/reports/XAUUSD/OOS/2026-09-06
verdict
DUDOSA       522
DESCARTAR    308
MANTENER       4
```

Cómo se lee esto:

- **La retención mediana es del 24%.** De cada estrategia sobrevive una cuarta parte de su Sharpe. Eso no es una anomalía de unas pocas: es lo que le pasa a la población entera.
- **308 se descartan**, y el motivo dominante no es perder dinero — todas ganaban — sino la concentración: 283 sacan más del 60% de su beneficio de un solo trimestre, y 175 de ellas pierden dinero fuera de ese trimestre.
- **Ninguna llega a `t` = 2.** El máximo es 1,94. Traducido: ni una sola de las 834 tiene un rendimiento fuera de muestra que se distinga de la suerte con el criterio habitual del 5%.

- **Las 4 que pasan todo retienen entre el 68% y el 74% de su Sharpe**, ganan los cinco años y no concentran. Y 3 de las 4 salen de la misma plantilla.

Qué NO te dice

- **No te dice que las 4 supervivientes funcionen.** Dice que son las únicas que no puedes descartar con estos datos. Con 834 candidatas, que 4 destaquen es aproximadamente lo que produce el azar: si pruebas suficientes cosas, algunas salen bien solas. La confirmación tiene que venir de un tramo que estas estrategias no hayan visto nunca — un walk-forward hacia delante.
- **La retención por sí sola no clasifica.** Una estrategia que retiene el 90% de un edge que ya era ruido no vale más que una que retiene el 10% de uno bueno. Por eso la sentencia se apoya en `t`, en los años y en la concentración, y la retención se informa pero no decide.
- **No corrige por el número de intentos.** Si generaste 50 000 estrategias para quedarte con 834, el rendimiento de las mejores está inflado por esa búsqueda, y este informe no lo descuenta.
- **No mira las operaciones, mira la curva de equity diaria.** Un Sharpe calculado sobre P&L diario no es idéntico al que muestra SQX en el databank, que se calcula por operación. Las conclusiones no cambian, los decimales sí.

Si algo falla

- **`StopIteration` al leer un `.sqx`** — ese archivo no tiene curva de equity guardada, casi siempre porque la estrategia nunca llegó a ejecutarse en un backtest completo. Reconstrúyela en SQX o sácala del databank.
- **Salen 0 estrategias** — la carpeta del databank está vacía en disco. Es lo normal si SQX no ha sincronizado todavía: abre el databank en la GUI y espera a que escriba.
- **`years_positive` sale siempre 1** — te has equivocado con `--split` o `--end` y el tramo fuera de muestra cabe en un solo año natural.

29. La puerta OOS — de un databank entero a una lista de supervivientes

Qué pregunta responde

Has construido un montón de estrategias en un periodo y las has reteseado en el siguiente. Como SQX solo admite **un spread y un slippage por backtest**, y estas ventanas son de años sobre un activo que cambia mucho de precio, eso son dos tareas distintas y por tanto **dos databanks**: el de construcción y el de retesteo. Ahora tienes esos dos y no sabes qué estrategias merecen tu tiempo. La puerta las pasa por una **cascada de cribas**: primero las baratas, que tumban mucho; al final las caras, que solo ven a las que llegan vivas. Sale una lista de supervivientes, y para cada muerta, **en qué criba murió y con qué número**.

La puerta los **junta por estrategia** —por su identidad, no por su nombre— y los juzga juntos.

No es un filtro de métricas. Un filtro de métricas ya lo hace SQX con sus condiciones de aceptación. Esto añade lo que SQX no mira: cuánto se degradó respecto al periodo de construcción, si bate a un mono que operase el mismo mercado, y si tus 200 supervivientes son en realidad 30 repetidas.

Cuándo lo usas, y cuándo no

Lo usas cuando un proyecto propio ha terminado su tarea de build y su retest OOS, y quieres saber qué queda antes de gastar días de máquina en el protocolo de robustez.

No lo uses para juzgar al generador. Si la tarea de build llevaba condiciones de aceptación sobre el periodo OOS, esa población ya fue filtrada por la muestra que la puerta va a juzgar, y las cribas salen casi inertes. Se ve en el ejemplo de abajo: en `XAUUSD/OOS` sobreviven 228 de 231, y no porque sean buenas, sino porque solo entraron las que ya habían pasado ese corte. Para leer el resultado como un juicio hace falta un databank cuyo OOS **no** fuese condición de aceptación.

No decide nada por ti. Escribe el veredicto; borrar es otro comando (página 27).

Antes de empezar

- **Los dos databanks** tienen que existir en el mismo proyecto, con sus `.sqx` en disco.
- La cosecha lanza **un solo** **`orderstocsv`** en el conductor (worker de 5060) para sacar las operaciones de los dos lados; las métricas y las curvas se leen del `.sqx` sin SQX. Si otra sesión tiene el conductor, el comando se niega en vez de esperar.
- **Desde la ventana no hace falta lanzarla**: al elegir un databank de construcción, la cosecha se hace sola si falta o está vieja (capítulo 58, «La carga automática»).
- Si el proyecto vive en un worker en vez de en el maestro, pásale `--role custodian`.
- Las barras M1 del feed tienen que estar en la librería (`python3 -m sqx.export.sync_bars --check`).

Cómo se emparejan las dos mitades

Por la **identidad** de la estrategia: el SHA-256 de su definición interna, que no cambia al reetestearla en otra ventana con otros costes (comprobado: 5 de 5 en un par real). **No por el nombre**, y esto no es teórico: `Strategy 17.8.29` existe en dos databanks de este mismo proyecto y son estrategias completamente distintas, de plantillas distintas.

De ahí salen tres grupos, y los tres importan:

grupo	qué es	qué hace la puerta
emparejadas	están en los dos databanks	las juzga
sin OOS	están en el de build y **no** en el de retesteo	**las descarta** (criba `presencia`)
solo en OOS	están en el de retesteo y no en el de build	las ignora y las cuenta en el manifest

El grupo del medio es el importante: si SQX tiró una estrategia del retesteo porque saltó una de sus banderas rojas —demasiados trades ambiguos, o cualquier otra— esa decisión ya está tomada, y la estrategia se va. Y como el emparejamiento ocurre **antes** de exportar nada, no se gasta ni un segundo de máquina en ellas.

Cómo se ejecuta

Son **dos comandos**: uno saca los datos de SQX una sola vez, el otro juzga y se puede repetir tantas veces como quieras sin volver a tocar SQX.

```
python3 -m studies.screening.gate.harvest --project XAU_ISOOS_ejemplo --databank Results --oos-databank OOS --role custodian
python3 -m studies.screening.gate.report  --project XAU_ISOOS_ejemplo --databank Results --feed XAUUSD_DukasM1_Infinox
```

`--databank` es siempre **el de construcción**, en los dos comandos: es el que da nombre a la carpeta de la cosecha, y el que define la población que entra.

`studies.screening.gate.harvest` — la cosecha

flag	obligatorio	qué hace
`--project`	sí	proyecto tal y como aparece en SQX
`--databank`	sí	el databank **de construcción** (IS), con sus espacios
`--oos-databank`	sí	el databank **de retesteo** (OOS)
`--role`	no	`conductor` o `custodian` si el proyecto vive en un worker; sin él, el maestro
`--limit`	no	una muestra aleatoria de N **parejas**, para una prueba
`--exports`	no	además de la cosecha, deja la **exportación de operaciones de cada databank** en `raw/<proyecto>/<databank>/<fecha>/`, sacada del mismo `orderstocsv`. Es lo que leen después `edgeCost`, `spread`, el mono y el resto de estudios del paso 8: con el flag no hace falta un segundo arranque de SQX para exportarlas. Sólo escribe la de un lado si la cosecha se llevó **todas** sus estrategias (con `--limit` no)

Toca SQX solo para las operaciones: copia los `.sqx`, lee sus métricas y curvas del propio fichero (las mismas columnas que la vista `Export Data View`, sin redondear) y lanza un único `orderstocsv` para los dos lados. 100 + 40 estrategias: **18 segundos** (antes, dos ciclos de arrancar y parar el conductor, unos minutos).

`studies.screening.gate.report` — el juicio

flag	obligatorio	qué hace
`--project`, `--databank`	sí	de qué cosecha lee (el databank **de build**); coge siempre la más reciente
`--feed`	sí	feed de SQX con el que se le pagan las barras al mono, p. ej. `XAUUSD_DukasM1_Infinox`
`--set`	no	cambia un umbral sin editar el fichero: `--set degradacion.min_t=1.65`
`--seed`	no	la semilla de los monos del filtro `mono`. Sin él se sortea una nueva en cada corrida, y `mono` y `familia` pueden cambiar en las estrategias que rozan el umbral; con él, `--seed 20261002`, dos corridas dan la misma scorecard byte a byte. La usada queda en `manifest.json` (`nulls_seed`), en la columna `mono_seed` de la scorecard y en la última línea que imprime

No toca SQX en absoluto. Sobre 231 estrategias tardó segundos: la cosecha se lee en 0,3 s y el mono cuesta 0,1 s por estrategia con 2.000 corridas nulas. Puedes reejecutarlo con otros umbrales todas las veces que quieras.

Dónde están los umbrales. Los números de las cribas no están en `config.yaml`: allí pone `ledger:gate.<criba>.<umbral>`, y el valor, con quién lo puso y cuándo, está en `ledger/thresholds.yaml`. Moverlo de verdad es editar esa fila; `--set` sólo prueba otro valor en una corrida.

Qué produce

La cosecha — `~/Desktop/AlgoData/harvest/<proyecto>/<databank de build>/<fecha>/`, fechada e inmutable. Todo va indexado por **identidad**, no por nombre:

fichero	qué es
`metrics.parquet`	una fila por estrategia y **cada métrica dos veces**: `[IS]` del databank de build y `[OOS]` del de retesteo. Más `strategy` y `strategy_build`, los dos nombres
`trades.parquet`	todos los trades de los dos lados, con una columna `sample` que dice de cuál viene cada uno
`equity.parquet`	la equity diaria de cada lado, igual con su `sample`
`missing_oos.csv`	las que están en el build y no en el retesteo: identidad y nombre
`manifest.json`	los dos databanks, la clave del join, y cuántas emparejaron

Los `.sqx` copiados y los CSV intermedios **se borran**: pesan diez veces más que el Parquet.

⚠ Si la cosecha imprime `AVISO: columnas (OOS) con datos`, ese databank **no** corrió una sola ventana: llevaba su propio corte IS/OOS dentro, y la puerta está tirando la mitad de sus números. Es el aviso de que le has dado un databank de otro tipo.

El juicio — `~/Desktop/AlgoData/reports/<proyecto>/<databank de build>/<fecha>/gate/`:

fichero	qué es
`scorecard.parquet`	una fila por estrategia y tres columnas por criba: el valor, si pasó y la nota; más las **columnas medidas** de abajo, que no juzgan a nadie
`verdict.csv`	para el databank **de retesteo** — la población que sigue adelante
`verdict_build.csv`	para el databank **de construcción** — el único que puede nombrar a las que SQX tiró del retesteo, porque allí no tienen nombre
`funnel.csv`	el embudo: por criba, cuántas entran, pasan y mueren
`gate.md` / `gate.html` / `gate.json`	el embudo con los umbrales que estaban puestos y por qué existe cada criba (hasta el 25-09, `resumen.md`); el `.json` es lo que pinta la ventana
`estrategias/<nombre>.html` / `.json`	la ficha de una estrategia: cada criba con su valor y su nota

Las columnas medidas (desde el 2026-10-02)

Además de las cribas, el scorecard lleva doce números por estrategia que **no eliminan a nadie**: se miden para todas las que están en el databank de retesteo, también para las que murieron en la primera criba, y son los que el piloto automático puede usar en sus reglas (`gate.scorecard.<columna>`).

columna	qué es	cómo se lee
`trade_t`	la t de la operación media fuera de muestra: media ÷ desviación × √(nº de operaciones)	cuántos errores estándar separa el beneficio medio por operación de cero. Por debajo de 1,65 no se distingue del azar ni siquiera sin corregir por haber probado miles
`drift_excess_t`	lo mismo, pero quitándole antes a cada operación **lo que el mercado habría pagado por estar dentro**: dirección × deriva media por barra del periodo × barras que duró × tamaño × valor del punto	la que hay que mirar. Una estrategia larga en un mercado que sube gana parte de su dinero solo por estar comprada; esta t mide lo que queda. Sirve igual para largos y cortos y para cualquier activo. Se corrige además por la autocorrelación entre operaciones seguidas
`oos_trades_per_year`, `is_trades_per_year`	operaciones por año en cada ventana	—
`oos_max_dd_r`, `is_max_dd_r`	la mayor caída de la equity diaria desde su máximo, **con el flotante dentro**, en R. SQX guarda de cada día su equity **más baja** (las posiciones abiertas en su peor momento), así que es la caída flotante, no la de operaciones cerradas	cuánto habría que aguantar en una cuenta fondeada
`oos_worst_day_r`, `is_worst_day_r`	el peor día de servidor de esa equity, flotante incluido, en R (negativo). Va del mínimo de un día al mínimo del siguiente	una aproximación a lo que la firma compara con su límite de pérdida diaria (ella mide desde el inicio del día); sirve para ordenar, no para certificar
`oos_worst_trade_mae_r`, `is_worst_trade_mae_r`	la peor pérdida flotante que llegó a tener una operación (MAE), en R (negativo)	la doctrina no lleva stop: aquí se ve cuánto vale de verdad «1R»
`oos_net_per_year_r`, `is_net_per_year_r`	beneficio neto cerrado por año, en R	—

R es el riesgo fijo por operación de la doctrina: 1.000 USD (`assets/_build.yaml`).

⚠ **La `t` de la criba `degradacion` cambió de escala el 2026-10-02.** Tenía un error de unidades que la comprimía y le ponía un techo (en cinco años no podía pasar de 3,2). Corregida, es la t normal de la curva diaria: lo que antes era 1,3 / 1,65 / 2,0 ahora es 1,42 / 1,92 / 2,55. Un scorecard anterior a esa fecha está en la escala vieja; no los mezcles. Y aun corregida, se calcula sobre la equity diaria con el flotante dentro, que es más ruidosa que las operaciones cerradas: sale algo más baja que `trade_t` para la misma estrategia.

Cómo se lee el resultado

Lo primero que se mira es el embudo que imprime por pantalla. Éste es un proyecto real con las dos tareas separadas — 120 estrategias construidas, 115 que llegaron al retesteo:

```
115 emparejadas + 5 sin OOS, ventana 2017-11-29 -> 2022-12-29

presencia      hard  entran   120 pasan   115 mueren     5
sanidad        hard  entran   115 pasan   112 mueren     3
estaticas      hard  entran   112 pasan    64 mueren    48
degradacion    hard  entran    64 pasan    45 mueren    19
forma          hard  entran    45 pasan    45 mueren     0
mono           hard  entran    45 pasan    45 mueren     0
familia        soft  entran    45 pasan     0 mueren     0
redundancia    soft  entran    45 pasan    45 mueren     0

45 de 120 sobreviven
```

Se lee de arriba abajo: **qué criba está haciendo el trabajo**. Aquí, dos:

criba	qué mató	qué dice
`presencia`	5	SQX las tiró del retesteo por sus propias banderas rojas. Comprobado: faltan también por nombre, no es un fallo de emparejamiento
`sanidad`	3	pocos trades o trades idénticos a otra
`estaticas`	**48 de 112**	el 43 % simplemente **pierde dinero** fuera de muestra, con el umbral más laxo que existe
`degradacion`	19	de las que ganan, casi un tercio no aguanta su propio error estándar
`mono`	0	con `max_p: 0.50`. Pero la mediana de p entre los 45 supervivientes es **0,10**, y **sólo 5 de 45 batirían al mono al 5 %**

Y esos 45 supervivientes conservan una **mediana del 45 % de su Sharpe** de construcción, que es lo que se parece a la realidad. Compáralo con una población que ya venía filtrada por su propio OOS (`XAUUSD/OOS`): allí el 99 % ganaba dinero, la retención mediana era **1,00** y el mono no mataba a nadie. Esos números no eran buenos, eran el filtro mirándose al espejo.

`redundancia` agrupa por la **estructura** de la estrategia — los bloques de su XML y su orden, ignorando los valores de los parámetros. Aquí los 45 supervivientes son **13 formas distintas, y una sola se lleva 24 de ellas**. No elimina a nadie, pero es lo que hay que saber antes de montar una cartera con "45 estrategias".

La puerta, sin embargo, marca en el `scorecard.parquet` la **medoide** de cada grupo (`medoide`, y `redundancia_grupo` con el grupo): la estrategia cuyas barras de entrada más se parecen (Jaccard medio) a las del resto de su grupo. El juez del paso 8 se queda con las de los N grupos mayores (`medoids` en criteria.yaml) y corta el resto con la razón «no es la medoide de su grupo».

Una criba que mata 0 está puesta demasiado laxa **o** la población ya venía filtrada por eso mismo. Distinguir las dos cosas es mirar la distribución de su columna en el `scorecard.parquet`.

La columna `reason` del `verdict.csv` dice de qué murió cada descartada:

```
Strategy 21.30.45,DESCARTAR,...,presencia = 0 | SQX no la dejo en el databank OOS
Strategy 21.17.26,DESCARTAR,...,degradacion = 0.7249 | t=0.58 anos+=3 conc=1.06
Strategy 18.22.27,DESCARTAR,...,sanidad = 300 | trades identicos a otra estrategia
```

Las dos últimas son el caso que justifica la criba `sanidad`: **estrategias con los trades byte a byte idénticos** bajo nombres y hashes de fichero distintos. No son dos observaciones, son una.

Un ejemplo completo

```
$ python3 -m studies.screening.gate.harvest --project XAU_ISOOS_ejemplo --databank Results \
      --oos-databank OOS --role custodian
Results: 120 · OOS: 115 · emparejadas 115 (0 por nombre) · sin OOS 5 · solo en OOS 0
bloque con datos: build=(IS) · retesteo=(OOS)
wrote /home/sergioguslw/Desktop/AlgoData/harvest/XAU_ISOOS_ejemplo/Results/2026-09-23

$ python3 -m studies.screening.gate.report --project XAU_ISOOS_ejemplo --databank Results \
      --feed XAUUSD_DukasM1_Infinox
115 emparejadas + 5 sin OOS, ventana 2017-11-29 -> 2022-12-29
presencia      hard  entran   120 pasan   115 mueren     5
...
45 de 120 sobreviven -> .../reports/XAU_ISOOS_ejemplo/Results/2026-09-23/gate
```

Probar un umbral más duro sin volver a tocar SQX:

```
$ python3 -m studies.screening.gate.report --project XAU_ISOOS_ejemplo --databank Results \
      --feed XAUUSD_DukasM1_Infinox --set mono.max_p=0.05 --set degradacion.min_t=1.65
```

Y aplicar el resultado dentro de SQX (página 27, con el install parado). Ojo con **cuál** de los dos veredictos aplicas a **cuál** databank:

```
$ python3 -m sqx.curate.apply_verdict --project XAU_ISOOS_ejemplo --databank OOS --role
custodian \
      --verdict ~/Desktop/AlgoData/reports/XAU_ISOOS_ejemplo/Results/2026-09-
23/gate/verdict.csv --apply
```

Qué NO te dice

- **No te dice que las supervivientes sean buenas.** Te dice que no murieron con los umbrales que tenías puestos. Hoy son deliberadamente laxos.
- **No deshace la preselección.** Si el build ya filtraba por OOS, la puerta mide ese filtro, no la estrategia. Lo avisa el `gate.md`, pero no lo puede arreglar.
- **No te dice por qué SQX tiró una estrategia del retesteo.** La criba `presencia` sabe que no está, no por qué. Eso está en el log del proyecto y en las banderas rojas de SQX.

- **No compara los costes de las dos tareas.** Cada mitad lleva el spread y el slippage que su tarea cobró, y eso es lo que quieres; pero si los pusiste mal en una de las dos, la degradación que mide la puerta es la de tus costes, no la del mercado.
- **El mono entra solo en las horas de la estrategia** (desde 2026-10-12): cada operación del mono entra a la misma hora del reloj del feed que la real, dentro de la misma ventana de calendario y de la sesión del mercado; así el mono tiene el mismo número de entradas por hora que la estrategia. Sustituye al sorteo uniforme y no hay interruptor.

- **La z del mono es la única regla estadística del paso 8** (dueño, 2026-10-12): `z_mono` (tu ganancia media por operación frente a la del mono con horas, en desviaciones típicas del mono) tiene que llegar a **$\Phi^{-1}(1 - 0{,}05/B)$**, donde B es el número de lotes cribados en oos1 en tu celda (activo × temporalidad × dirección × familia) contando el actual: **1,645 con B = 1, 1,960 con 2, 2,241 con 4**. Con más lotes probados en la misma celda, la vara sube, porque con más intentos es más fácil que el azar produzca una z alta. `criteria.yaml` no lleva el número, solo `alpha: 0.05`; el juez lo calcula, y la fila del ledger apunta B y el umbral usado. R3, la t de R5c y la antigua línea fija del mono ya no juzgan; R4a, R5, R5b, R8 y R9b siguen calculándose y guardándose en el scorecard como columnas, sin cortar a nadie. Sin dato de `z_mono` la estrategia cae.

- **`p_fdr`: la probabilidad de que no sea una estrategia sin ventaja (todavía no corta).** Con las z de todas las estrategias que llegaron al mono se mide cómo se comporta «el azar» *en ese lote* (su centro: media δ_0, anchura σ_0) y qué parte del lote parece puro azar (π_0). `p_fdr` es, para cada estrategia, la probabilidad que sale de comparar su z con ese azar: cerca de 0, tu z es de las que el lote produce sola; cerca de 1, casi seguro hay algo más. Vale 0 si tu z está en o por debajo del centro del azar, y también si la cola del lote es más ligera que su propio azar: entonces nadie destaca y todas valen 0. Con menos de 500 z no se calcula (sería ruido) y la columna queda vacía. **Hoy no corta a nadie**: solo se muestra y ordena el scorecard (de más a menos `p_fdr`); dónde cortar se decide tras el build sintético (fila 9 del encargo 65). δ_0, σ_0 y π_0 salen en la nota del embudo.

- **El mono sortea 100.000 veces por estrategia** (antes 5.000; `monkey.draws` en `config.yaml`): el error de Monte Carlo de la z es ≈ 0,005. En la exportación de USATEC (169 estrategias vivas) el nivel del mono tardó **149 s**; como R3 ya no corta antes, llegarán más estrategias a este nivel y cada una cuesta ≈ 0,9 s.
- **El mono solo lee un escalón y un estadístico.** El escalón `timing` aleatoriza *cuándo* entra cada trade y nada más, y el estadístico por defecto es `sharpe`. Con `net` el mismo corpus da un resultado muy distinto (51 % contra 21,5 % de supervivientes, medido el 2026-09-22). El estadístico mueve el veredicto más que el nulo: página 26.
- **`redundancia` no dice que dos estrategias correlacionadas sean la misma.** Dice que a efectos de cartera aportan casi lo mismo durante ESE periodo.
- **Nada de esto sustituye al protocolo de robustez** (páginas 15, 17, 18, 19): la puerta elige a quién dedicarle esos días de máquina.

Si algo falla

- `KeyError: 'custodian'` — le has pasado un `--role` que no está en `config/machine.yaml`.
- `emparejadas 0` — los dos databanks no comparten ninguna estrategia. O te has equivocado de databank, o las del retesteo se regeneraron y ya no son las mismas: la identidad es la definición, y una estrategia reconstruida es otra estrategia.

- `KeyError: 'OOS'` al cargar la cosecha — esa cosecha es del formato viejo, de un solo databank. Vuelve a cosechar.
- El embudo dice `entran 0` en `estaticas` — la consulta de `keep` nombra una columna que esa vista no exporta. Mira las columnas reales en el `metrics.parquet` de la cosecha.
- `SystemExit` dentro del mono, hablando de `max_hold` — alguna estrategia mantiene una posición más barras de las que el escáner mira hacia delante. Es una negativa a propósito, no un fallo: subir `barrier.max_hold` en `engines/nulls/config.yaml` es la decisión que hay que tomar a mano.
- `REFUSING: another sqcli already holds SQX_w1` — otra sesión está usando el conductor. Espera a que acabe y repítelo (en la ventana, el botón ↻ de la barra de carga).

El contador por celda — `python3 -m ledger.cells`

Una celda es activo × temporalidad × dirección × familia de idea. Para cada una el contador lleva seis enteros: plantillas probadas, lotes construidos, estrategias generadas, estrategias cribadas en oos1, ediciones de `criteria.yaml` desde el último sello y miradas a oos2. Lo escriben el constructor de proyectos (al crear) y el juicio del paso 8; un proyecto `Test_` cuenta igual que uno `Trade_`. El dato es `AlgoData/ledger/cells/projects.csv`, una fila por proyecto, así que repetir una escritura no cuenta dos veces. `B` (lotes cribados en oos1 de la celda) es lo que leerá la regla del paso 8.

El comando se ejecuta **una vez**: reconstruye el contador desde `projects/registry.csv` y desde los `gate/funnel.csv` de `AlgoData/reports/`, y imprime cuántos proyectos tienen familia desconocida (van a la celda `unknown`) y las celdas con B ≥ 2. No toca SQX.

El sello (`python3 -m ledger.cells --seal`) se pone una sola vez, al cierre del bloque B, y sella todas las celdas a la vez con el sha256 de `criteria.yaml` de ese momento. Desde entonces, «ediciones desde el último sello» de una celda es el número de hashes distintos del sellado entre sus juicios del paso 8 posteriores (`AlgoData/ledger/cells/judgements.csv`). Antes del sello la columna está vacía. Solo cuenta: no rechaza ninguna corrida.

¿Se probó esta celda? (`python3 -m ledger.cells --ask SÍMBOLO MARCO [--template T] [--direction long|short] [--json]`) es sólo lectura: junta los tres registros (`projects/registry.csv`, `campaign/ledger.jsonl` y el libro de búsqueda) y dice, para la celda, los proyectos construidos por tipo (`test`, `trade`, `research`), el estado de campaña, y por tramo (`build`, `oos1`, `oos2`) las filas del libro, las cribas del paso 8, los días y los criterios distintos; lo mismo sumado sobre la serie (símbolo × marco) y sobre el símbolo entero. Un `Test_` gasta tramos pero no la celda (dueño, 2026-10-14). Veredicto: `sin probar`, `construida, sin juzgar`, `oos1 gastado N veces` u `oos2 abierto`.

98. Criba del MAE profundo — de qué operaciones vive una estrategia

Qué pregunta responde

La cadena construye sin stop a propósito. El precio: la búsqueda puede premiar estrategias que ganan **porque nunca cortan**. Una operación que se fue 5 ATR en contra y volvió cuenta como una ganadora más; el día que se le ponga un stop, deja de serlo. Esta criba, en el paso 8, mide cuánto del resultado de cada estrategia vive de esas operaciones, cuando descartar todavía es barato. No elige ningún stop: avisa.

Cuándo lo usas, y cuándo no

Lo usas tras la puerta (`studies.screening.gate.report`), sobre su misma cosecha. **No** sirve para elegir el stop (eso ya no es un paso del workflow), ni para juzgar `oos2` (nunca lo lee: sólo `build` y `oos1`). Si la estrategia ya lleva un stop, no hay nada que aguantar.

Antes de empezar

La cosecha de la puerta tiene que existir (`AlgoData/harvest/<proyecto>/<databank>/<día>/`) y las barras del feed, en `AlgoData`. No toca SQX: se puede lanzar con cualquier instalación abierta.

Cómo se ejecuta

```
python3 -m studies.screening.maeDepth.report --project Test_USDJPY_donchianUpperCrossUp_M30 --
databank Results --feed USDJPY_M1 --family <familia>
```

flag	obligatorio	qué hace
`--project`	sí	proyecto de la cosecha
`--databank`	sí	el databank de build que leyó la puerta
`--feed`	sí	el feed M1 del mercado, p. ej. `USDJPY_M1`
`--family`	no	familia de plantilla para el ledger; sin ella no se escribe la fila del ledger y el informe lo dice
`--set`	no	`sección.clave=valor`, p. ej. `--set anomaly.sigma=2`

Tarda unos segundos (21 estrategias, 34.118 operaciones: 2 s). Una línea `PROGRESS` por avance.

Qué produce

En `AlgoData/reports/<proyecto>/<databank>/<día>/maeDepth/`: `maeDepth.json/.html/.md` (la población), `estrategias/<nombre>.json/.html` (cada una, `build` y `oos1` lado a lado, y el histograma del MAE en ATR de ganadoras y perdedoras), `panel.parquet` (todas las medidas) y `verdict.csv` (`strategy`, `identity`, `verdict`) que `/curate` sabe aplicar.

Cómo se lee el resultado

```
$ python3 -m studies.screening.maeDepth.report --project Test_USDJPY_donchianUpperCrossUp_M30 --databank Results --feed USDJPY_M1
corte de duración (media + sigma desviaciones, en barras) y cuántas operaciones lo pasan
estrategia                                  build                                oos1
                      media  desv.  corte  n>=corte    n   media  desv.  corte  n>=corte    n
Strategy 9.13.86       17.2    2.7   24.0         0  1127    17.4    2.6   23.9         0   616
Strategy 5.19.75       14.4    2.2   19.9         0   954    14.5    2.1   19.9         0   497
Strategy 9.9.85        17.3    2.7   24.0         0  1219    17.4    2.7   24.1         0   638
Strategy 5.11.48       14.4    2.2   19.8         0   781    14.5    2.2   19.9         0   433
Strategy 14.13.63      18.1    3.0   25.5         0   581    18.5    2.5   24.8         0   352
Strategy 4.10.81       17.3    2.6   23.8         0  1365    17.3    2.8   24.4         0   735
Strategy 5.13.71       14.4    2.2   19.8         0   767    14.5    2.1   19.9         0   426
Strategy 19.15.62      10.8    1.2   13.7         0   589    10.7    1.4   14.2         0   329
Strategy 13.14.82      14.5    1.9   19.3         0  1344    14.6    2.0   19.6         0   725
Strategy 5.9.56        14.5    1.9   19.3         0  1214    14.6    2.0   19.6         0   690
Strategy 15.1.63       17.3    2.5   23.6         0  1238    17.4    2.5   23.7         0   687
Strategy 4.12.48       17.3    2.6   23.8         0  1367    17.3    2.8   24.4         0   735
... 21 estrategias; con corte > 0: build 0, oos1 0
21 estrategias, 34118 operaciones en 1.4 s; sin umbral fijado: ninguna marcada; sin fila de ledger (falta --family) -> /home/serg…
```

Para cada operación: el MAE en ATR(20) de SQX de la barra cerrada antes de la entrada. Por estrategia y tramo, para cada k (3 y 4 ATR por defecto):

- **profundas**: operaciones con MAE ≥ k ATR, y su fracción.
- **recuperadas**: las profundas que cerraron en ganancia neta, y la **cuota** del beneficio neto del tramo que dan. Si el neto es ≤ 0 no hay cuota. Puede pasar de 1: si el neto es pequeño, las recuperadas lo explican varias veces.
- **anómalas**: profundas y además de duración ≥ media + 2,5 desviaciones de esa estrategia en ese tramo, en barras (no horas). Es el retrato de aguantar una perdedora hasta que vuelve.
- **neto cortado**: lo que quedaría si cada profunda se hubiera cerrado en −(k + deslizamiento) ATR, con deslizamientos 0, 0,25 y 0,5. Es una **cota, no un backtest de stop**: un stop real se llena con hueco.

La tabla impresa enseña el **corte de duración** de cada estrategia antes de cruzarlo con el MAE. Una estrategia cuyas recuperadas pesan poco en `build` y mucho en `oos1` es una señal por sí misma.

Un ejemplo completo

La captura de arriba es la cosecha `Test_USDJPY_donchianUpperCrossUp_M30` (21 estrategias). Lectura real: en `build` las profundas (≥ 3 ATR) son de media el 26 % de las operaciones y las recuperadas dan de media el 33 % del neto (de 10 % a 52 % según la estrategia); en `oos1` bajan al 8 %. Y el corte de duración **no marca ninguna operación en ninguna estrategia**: esas estrategias salen «tras X barras», su duración casi no varía y media + 2,5 desviaciones queda por encima de todas. Con ese corte, «anómala» no se cumple nunca en esa población.

Qué NO te dice

No dice qué stop poner ni que vaya a funcionar. No marca a nadie mientras el umbral (`verdict.min_cuota_rec`) esté sin fijar: es decisión del dueño. El contrafactual no incluye costes de salida más allá del deslizamiento.

Si algo falla

- `ningún fichero de assets/symbols/ declara el feed`: el feed no pertenece a ningún activo.
- Una estrategia sin ganadoras o sin neto positivo no tiene cuota ni percentil: sale vacía, no es un error.

99. Monos por la cadena — ¿cuántas sin ventaja se cuelan, y cuántas buenas se pierden?

Qué pregunta responde

«Si de diez mil monos llegan 4 y de diez mil tuyas llegan 5, ya sabes lo que vale ese 5.»

Coge una población real, y sobre la huella de sus estrategias sortea **monos**: listas de operaciones con el mismo número de operaciones y el mismo coste que su donante, pero con la entrada puesta al azar sobre las mismas barras. Con ellos hace **dos poblaciones de prueba**:

- **La nula**: los monos tal cual. No tienen ventaja ninguna; las que pasan, pasan por suerte. Es la **tasa de falsos positivos** de la cadena y de cada regla.
- **Las de ventaja plantada**: los mismos monos, a los que se suma en cada operación de oos1 una ganancia fija de **0,05, 0,10 y 0,15 de Sharpe por operación, neto de costes**: ese número × la desviación del exceso de ese mono, más lo que ese mono paga de coste medio por operación. Así, una población «0,15» gana de media 0,15 **después** de los costes que cobra la puerta; la pregunta que responde es «si una estrategia gana 0,15 neto, ¿la ve la puerta?». Son estrategias buenas *por construcción*; las que caen son buenas que la cadena pierde, y dice **en qué regla**.

A las cuatro les aplica **las mismas reglas que el autopilot aplica a las de verdad** (`pipeline/autopilot/criteria.yaml`), y al lado pone a la población real, pasada por el mismo código. El paso 8 es la puerta entera, **con la cascada apagada**: cada regla se calcula para todas, no sólo para las que siguen vivas, que es la única forma de ver lo que hace cada una por separado.

Cuándo lo usas, y cuándo no

Lo usas antes de fijar las líneas de la puerta, y cada vez que cambies una regla de `criteria.yaml`: la pregunta deja de ser «¿qué umbral parece razonable?» y pasa a ser «a 1,65 pierdo el X % de las de 0,10 y dejo pasar el Y % de las nulas».

No lo usas para decidir sobre una estrategia concreta: no juzga a ninguna real y no escribe `verdict.csv`. Y **no mide el generador de SQX**: el mono nace en Python y nadie lo ha seleccionado en un build. Los monos construidos y elegidos por SQX son otro estudio (encargo 30).

Hoy sólo aplica el paso 8. Los pasos 10, 12 y 14 no tienen mono todavía y dejan pasar a todos; desde el paso 10 la cifra de vivos es un **techo**, no una medida.

Antes de empezar

- La **cosecha** de la población real (la misma que lee la puerta OOS; skill `/oos-gate`), en `AlgoData/harvest/<proyecto>/<databank>/<fecha>/`.
- Las barras M1 del activo en `AlgoData/bars/`, y sus costes declarados en la ficha del activo.
- No toca SQX: se puede lanzar con cualquier instalación abierta. **Sí carga la máquina**: el mono de la puerta usa todos los núcleos; no lo lances al lado de un workflow.
- La cosecha tiene que ser **posterior al cambio de reloj de los feeds (2026-10-01)**. Con una anterior las operaciones no casan con las barras y el estudio deja fuera a todas las donantes.

Cómo se ejecuta

```
python3 -m studies.screening.falsePositives.report \
    --project Test_USA500_H1_indicesIBS --databank Results --feed USA500_M1 \
    --harvest ~/Desktop/AlgoData/harvest/Test_USA500_H1_indicesIBS/Results/2026-10-05
```

flag	obligatorio	qué hace
`--project`	sí	el proyecto de la población real
`--databank`	sí	el databank bajo el que está su cosecha (normalmente `Results`)
`--feed`	sí	el feed de SQX del activo, p. ej. `USA500_M1`
`--harvest`	no	la carpeta de la cosecha que se lee. **Pásala siempre que haya más de una**: sin ella coge la más reciente del databank, que tras un corte puede ser la de tres supervivientes
`--criteria`	no	otro fichero de reglas con la forma de `criteria.yaml`. Sin él, el del autopilot. Sirve para probar una línea **antes** de ponerla
`--seed`	no	la raíz de todos los sorteos (monos, permutaciones, el mono de la puerta). Sin ella usa `population.seed`, y si está vacía sortea una nueva; la usada queda en la última línea y en el manifiesto. Con la misma raíz la corrida sale igual
`--ledger`	no	apunta en el registro una fila por paso juzgado, de la población nula. Úsalo en la corrida buena, no en las pruebas
`--set population.per_level=5000`	no	estrategias por población. Se sortean `per_level / per_donor` donantes de la cosecha
`--set monkeys.per_donor=50`	no	monos por donante. 100 donantes × 50 = 5.000
`--set population.edges=...`	no	los niveles de ventaja plantada; por defecto 0,05, 0,10 y 0,15
`--set monkeys.rung=free`	no	qué conserva el mono de su donante: `free` sólo el número de operaciones y el coste; `timing` también la duración y el tamaño de cada operación
`--set monkeys.sizing=atr`	no	cómo se dimensiona el mono: `atr` con la volatilidad de la barra anterior a la entrada, como tu doctrina (por defecto); `donor` con el tamaño de la donante

Lo que tarda. La cadena entera se calcula una vez por población (cuatro) más una por la real. Con 60 monos por población y 10 reales, 34 segundos en cuatro núcleos. Lo caro es el mono de la puerta (5.000 sorteos por estrategia, unos 0,23 segundos de núcleo cada una): 5.000 por población son 20.000 estrategias, unos 20 minutos en cuatro núcleos sólo de eso.

Qué produce

En `AlgoData/reports/<proyecto>/<databank>/<día>/falsePositives/`:

fichero	qué es
`falsePositives.json` / `.html` / `.md`	el resultado entero, el que pinta la ventana
`funnel.csv`	el embudo paso a paso, una fila por población (`nivel`) y paso
`reglas.csv`	**regla × población**: cuántas deja en Pasa, Limbo y Falla ella sola (`pass`, `limbo`, `fail` y sus proporciones `pct_*`), cuántas caen en ella las primeras (`died`, en el orden de la puerta) y `nota`, lo que esa cifra no es
`hechos_monos.parquet`	los hechos de cada estrategia de cada población (los del scorecard de la puerta)
`manifest.json`	la cosecha leída, el fichero de reglas, la semilla y el comando

Una corrida del mismo día sobrescribe la anterior.

Cómo se lee el resultado

```
$ python3 -m studies.screening.falsePositives.report --project Test_NIKKEI225_M30_simpleL_fin \
      --databank Results --feed NIKKEI225_M1 --harvest ~/Desktop/AlgoData/harvest/Test_NIKKEI225_M30_simpleL_fin/Results/2026-10-05 \
      --seed 9 --set population.per_level=60 monkeys.per_donor=6
 nivel step        estado  n_in  pass  limbo  n_out
  nula    8      aplicado    60     0      0      0
  nula   10 0 de 2 reglas     0     0      0      0
  nula   12 0 de 1 reglas     0     0      0      0
  nula   14 0 de 1 reglas     0     0      0      0
  nula   16     sin regla     0     0      0      0
  0.05    8      aplicado    60     0      1      1
  0.05   10 0 de 2 reglas     1     1      0      1
  0.05   12 0 de 1 reglas     1     1      0      1
  0.05   14 0 de 1 reglas     1     1      0      1
  0.05   16     sin regla     1     1      0      1
  0.10    8      aplicado    60     0      8      8
  0.10   10 0 de 2 reglas     8     8      0      8
  0.10   12 0 de 1 reglas     8     8      0      8
  0.10   14 0 de 1 reglas     8     8      0      8
  0.10   16     sin regla     8     8      0      8
  0.15    8      aplicado    60     0     14     14
  0.15   10 0 de 2 reglas    14    14      0     14
  0.15   12 0 de 1 reglas    14    14      0     14
  0.15   14 0 de 1 reglas    14    14      0     14
  0.15   16     sin regla    14    14      0     14
reales    8      aplicado  1000     0      0      0
reales   10 0 de 2 reglas     0     0      0      0
reales   12 0 de 1 reglas     0     0      0      0
reales   14 0 de 1 reglas     0     0      0      0
reales   16     sin regla     0     0      0      0
paso 10 · 0 de 2 reglas: el mono en los otros mercados (su huella del retest, colocada sobre las barras de cada mercado) no está construido
paso 12 · 0 de 1 reglas: las hermanas escaladas las fabrica SQX reescalando parámetros, y un mono no tiene parámetros; además la regla nunca da Fa
paso 14 · 0 de 1 reglas: el estrés combinado del MC Retest es una tarea de SQX sin modelo en Python
60 monos de 10 donantes (0 fuera) · neto medio OOS +0.30 R/año · 53.3% con neto OOS positivo
semilla 9 -> /home/sergioguslw/Desktop/AlgoData/reports/Test_NIKKEI225_M30_simpleL_fin/Results/2026-10-07/falsePositives
```

- **`nivel`**: la población. `nula` no lleva ventaja; `0.05`, `0.10` y `0.15` son los mismos monos con esa ventaja plantada; `reales` es la población de verdad.
- **`n_in`** → **`n_out`**: cuántas entran al paso y cuántas salen. `n_out` = pass + limbo, porque un limbo sigue adelante, igual que en el autopilot.
- **`estado`**: `0 de 2 reglas` quiere decir que las reglas de ese paso no se han podido aplicar al mono; debajo de la tabla dice por qué.
- **La línea de dinero**: neto medio de la población nula. Si gana (aquí +0,30 R al año) no es que se le haya regalado ventaja: un mono **largo sobre un índice** se lleva la deriva del mercado. Por eso la regla R3 mide el exceso sobre esa deriva y no el neto.

> **Las dos capturas son anteriores al 2026-10-07**, cuando la ventaja se plantaba *encima* de un mono que ya pagaba costes (un «0,15» era entonces 0,15 menos el coste). Las columnas `nula` y `reales` siguen valiendo; las de `0.05`, `0.10` y `0.15` salen ahora más altas. Se rehacen con la próxima corrida.

En este ejemplo, hecho con la ventaja a la antigua (60 monos por población, sobre las 10 estrategias de un proyecto del Nikkei en M30): de las nulas no cruza ninguna el paso 8; con 0,05 cruza 1, con 0,10 cruzan 8 y con 0,15, **14 de 60**. Es decir: de estrategias con una ventaja de 0,15 por operación *antes* de costes, esta cadena y estos costes dejaban pasar a menos de una de cada cuatro. Sesenta son muy pocas para fiarse de la cifra; es la forma de leerla lo que enseña el ejemplo.

```
pass, cada regla sola:
                              regla   nula   0.05   0.10   0.15  reales
               presencia_passed >= 1 100.0% 100.0% 100.0% 100.0%    1.0%
                 sanidad_passed >= 1 100.0% 100.0% 100.0% 100.0%    1.0%
                 estaticas_value > 0  53.3%  73.3%  90.0%  95.0%    0.1%
           is_trades_per_year >= 35   0.0%   0.0%   0.0%   0.0%    0.0%
               sanidad_value >= 100 100.0% 100.0% 100.0% 100.0%    1.0%
             drift_excess_t >= 1.65   0.0%   1.7%  13.3%  30.0%    0.0%
                      z_frec <= 1.5 100.0% 100.0% 100.0% 100.0%    1.0%
                     z_frec >= -1.5 100.0% 100.0% 100.0% 100.0%    1.0%
           edge_mean_oos t_exceso_k   0.0%   0.0%  11.7%  28.3%    0.0%
                         e_fin t_b  95.0%  95.0%  95.0%  95.0%    1.0%
t_sin_3_mejores t_sin_mejor_trimestre   0.0%   0.0%   0.0%  10.0%    0.0%
                    mae_p95_r <= 1.5   0.0%   0.0%   0.0%   0.0%    0.0%
          n_afectadas ret_datos t_a  13.3%  25.0%  38.3%  60.0%    0.0%
     retraso_ret_min retraso_t_delta  25.0%  35.0%  63.3%  83.3%    0.0%
                         q_p > 0.05  93.3%  93.3%  93.3%  93.3%    1.0%
                        dd_p > 0.10 100.0%  98.3%  98.3%  96.7%    1.0%
                     z_mono >= 1.65   0.0%  13.3%  30.0%  43.3%    0.0%
```

La tabla «pass» dice qué proporción de cada población da por buena **esa regla sola**:

- En la columna `nula` es lo que se cuela. Una regla que da por buena a toda la nula (100 %) no protege del azar.
- En las columnas plantadas es lo que se conserva. Una regla que da por buena a la misma proporción en todas las columnas **no distingue** a una buena de una nula.
- En la captura (ventaja a la antigua) R3 (`drift_excess_t >= 1.65`) da por buena al 0 % de la nula y sólo al **30 %** de las de 0,15: es la regla que más buenas se lleva. Con la ventaja neta, esa cifra ronda el 60 % (tabla de abajo).

La tabla «fail» es lo mismo para las que elimina. Lo que no es ni Pasa ni Falla es Limbo, y sigue.

Por qué no 67 %. Con 196 operaciones, una ventaja de 0,15 sube la t en 0,15 × 14 = 2,1, y de libro pasarían la línea de 1,65 dos de cada tres (67 %); una nula, el 5 %. Las dos cifras son **sin costes y con la t sencilla**. La puerta mide otra cosa, a propósito:

qué se mide	nula que pasa R3	de 0,15 que pasa R3
de libro: t sencilla, sin costes	5 %	67 %
la t de la puerta, **sin costes** (prueba sintética, 1.000 monos de 196 operaciones)	4,1 %	62 %
la t de la puerta, **con un coste que le quita 0,8 de t** (misma prueba)	0,9 %	62 %
poblaciones reales sin ventaja, medidas el 2026-10-06	4,05 % sin costes · 0,87 % con ellos	—

La t de la puerta corrige por operaciones que se pisan y por colas gordas, y eso sólo puede bajarla. El mono de la población nula **paga los costes del activo**, y por eso cruza menos (0,9 % en vez de 4,1 %). La ventaja plantada es **neta**: a cada operación se le devuelve el coste medio del mono y se le suma la ventaja, así que el coste no le quita nada y pasa el mismo 62 % con costes que sin ellos. En la misma prueba, con costes: de 0,05 pasa el 13 %, de 0,10 el 34 % y de 0,15 el 62 %. La suerte de cada mono se conserva: por eso no pasan todas.

Un ejemplo completo

```
python3 -m studies.screening.falsePositives.report --project Test_NIKKEI225_M30_simpleL_fin \
    --databank Results --feed NIKKEI225_M1 \
    --harvest ~/Desktop/AlgoData/harvest/Test_NIKKEI225_M30_simpleL_fin/Results/2026-10-05 \
    --seed 9 --set population.per_level=60 monkeys.per_donor=6
```

La salida es la de las dos capturas. Las `reales` salen como 1.000 porque ese build tuvo 1.000 estrategias y el retest sólo conserva 10: las otras 990 fallan la regla de presencia.

Qué NO te dice

- **No es la cadena entera.** Sólo el paso 8 está aplicado.
- **Hay reglas que el mono no mide**, y `reglas.csv` lo dice en `nota`:
- *Presencia, clones, operaciones por año y estabilidad de la frecuencia*: el mono tiene, por construcción, las mismas operaciones que su donante. Su resultado es el de la donante.
- *R5 y R5d* (frecuencia y degradación dentro de oos1) se centran sobre la población; con la cascada apagada se centran sobre todas, no sobre las que llegarían a ese nivel. **Sólo diagnóstico.**
- *Retraso de una barra y calidad de datos*: la retención sólo existe cuando el exceso medio es positivo. En la nula, media población no tiene dato y queda en Limbo.
- **La ventaja plantada no depende de la barra**: es la misma cantidad en cada operación, así que sobrevive entera al retraso de una barra. Una ventaja de verdad puede vivir en la barra de entrada y perderse al retrasarla; ésa no es la que se planta aquí.
- **La ventaja se planta sólo en oos1.** La mitad in-sample del mono no la lleva (los sorteos al azar no se leen nunca sobre el IS).
- **La curva del mono es de operaciones cerradas**; la de SQX es el mínimo de cada día con el flotante dentro. El peor día y el drawdown del mono salen algo más suaves que los de una real.
- **Todas las reglas son provisionales** mientras `criteria.yaml` no esté congelado; el resultado lo avisa arriba.

Si algo falla

- **`No objects to concatenate`**: ninguna donante reconcilia contra las barras (correlación por debajo de 0,99). Casi siempre es una cosecha anterior al cambio de reloj de los feeds: hay que volver a cosechar.
- **`a IS trade has no ATR to size by`**: una operación cae en las primeras barras del fichero, donde todavía no hay ATR. Las barras tienen que empezar antes que la ventana de build.
- **`nulls: un trade dura N barras y barrier.max_hold es 400`**: una operación real dura más que el techo del motor de monos; se sube en `engines/nulls/config.yaml`, no se trunca.
- **Sale una población mucho menor que `per_level`**: la cosecha tiene pocas donantes (mira «Donantes fuera») o has leído la cosecha de después del corte. Pasa `--harvest`.

108. Lotes de una celda — ¿puedo sumar varios builds como si fuesen uno grande?

Qué pregunta responde

La máquina no puede generar 100.000 estrategias de una vez, así que se generan en lotes de 10.000 de la misma celda (misma plantilla, mismo activo, mismo timeframe). Esta prueba dice si esos lotes se pueden contar juntos como un único build grande, o si algo cambió entre ellos o ya se están repitiendo las mismas estrategias.

Mira tres cosas, en este orden:

1. **Que la configuración sea la misma**: plantilla, activo, timeframe, feed, tramos y las reglas del paso 8. Si algo difiere, lo nombra y no mira nada más.
2. **Que los embudos del paso 8 no difieran más que el azar**: cuántas mueren por cada regla y cuántas sobreviven, lote contra lote.
3. **Que el lote nuevo no repita lo que ya había**: cuántas estrategias del lote ya estaban, idénticas, en los anteriores.

Cuándo lo usas, y cuándo no

Después de correr dos o más veces el workflow en la misma celda, antes de juntar sus resultados o de decidir si lanzar otro lote. No sirve para comparar celdas distintas (dirá «configuración distinta»), ni para decidir si una estrategia es buena: solo dice si los lotes son la misma población.

Antes de empezar

Cada lote tiene que haber pasado el paso 8 del workflow: la prueba lee el informe de la puerta (`gate`) de cada proyecto. No toca SQX; se puede lanzar con cualquier cosa abierta. Un proyecto ya retirado sirve igual, porque sus informes se quedan en `AlgoData/reports/`.

Cómo se ejecuta

```
python3 -m studies.screening.batches.report Test_USDJPY_M30_doubleROCLong_10k
Test_USDJPY_M30_doubleROCLong_10k_B2 --seed 1
```

argumento	obligatorio	qué hace
los proyectos	sí, dos o más	los lotes, del más antiguo al más nuevo; las repetidas de cada lote se cuentan contra todos los anteriores
`--seed`	sí	la semilla del sorteo de la prueba de homogeneidad; se guarda en `batches.json` para poder repetir el resultado exacto

Tarda unos segundos. Acaba con código 0 si dice SUMAR o AVISO, y 1 si dice PARAR CELDA o NO SUMAR.

Qué produce

En `AlgoData/reports/<último proyecto>/Results/<día>/batches/`:

- `batches.md`: el informe que imprime.
- `batches.json`: el veredicto, las cifras y de qué informes de la puerta se leyó cada lote.

Si se lanza dos veces el mismo día sobre los mismos lotes, sobrescribe esos dos archivos.

Cómo se lee el resultado

```
$ python3 -m studies.screening.batches.report Test_USDJPY_M30_doubleROCLong_10k Test_USDJPY_M30_doubleROCLong_10k_B2
# Lotes de una celda — Test_USDJPY_M30_doubleROCLong_10k + Test_USDJPY_M30_doubleROCLong_10k_B2

**SUMAR** — no se detecta diferencia entre lotes ni repetidas (con pocos lotes, potencia baja). Generadas acumuladas: 20,000.

## ¿Se comportan igual los lotes?

Ji-cuadrado entre lotes 127.7; p = 0.7363 barajando islas enteras (5,000 veces), alarma por debajo de 0.01. Las islas de un mismo lote se dispersan 50.1 veces lo que darían sorteos independientes: po…

| lote | generadas | sobreviven al 8 | en cuántas islas | la isla que más lleva |
|---|---|---|---|---|
| Test_USDJPY_M30_doubleROCLong_10k | 10,000 | 56 | 12 de 20 | isla 12: 19 |
| Test_USDJPY_M30_doubleROCLong_10k_B2 | 10,000 | 48 | 14 de 20 | isla 11: 11 |

Por regla, de las que llegan a ella (diagnóstico, no prueba: con estrategias emparentadas estas z salen más grandes que con el azar solo):

| regla | mueren Test_USDJPY_M30_doubleROCLong_10k | mueren Test_USDJPY_M30_doubleROCLong_10k_B2 | z Test_USDJPY_M30_doubleROCLong_10k_B2 |
|---|---|---|---|
| sanidad_passed >= 1 | 299 de 10,000 (3.0%) | 127 de 10,000 (1.3%) | -8.4 |
| estaticas_value > 0 | 4,240 de 9,701 (43.7%) | 4,002 de 9,873 (40.5%) | -4.5 |
| is_trades_per_year >= 35 | 1,820 de 5,461 (33.3%) | 1,662 de 5,871 (28.3%) | -5.8 |
| drift_excess_t >= 1.65 | 3,545 de 3,641 (97.4%) | 4,120 de 4,209 (97.9%) | +1.5 |
| e_fin t_b | 0 de 96 (0.0%) | 1 de 89 (1.1%) | +1.0 |
| mae_p95_r <= 1.5 | 36 de 96 (37.5%) | 35 de 88 (39.8%) | +0.3 |
| dd_p > 0.10 | 4 de 60 (6.7%) | 5 de 53 (9.4%) | +0.5 |

## ¿Estoy generando lo que ya tenía?

Aviso a partir del 10% del lote, parar la celda desde el 30%.

| lote | repetidas | del lote | supervivientes repetidas | la celda tiene al menos |
|---|---|---|---|---|
| Test_USDJPY_M30_doubleROCLong_10k_B2 | 0 de 10,000 | 0.0% | 0 de 48 | > 100,000,000 distintas |

La huella es exacta: dos estrategias con el periodo 14 y 15 cuentan como distintas. No ve un error común a todos los lotes, ni las casi gemelas.
```

La primera línea es la decisión:

dice	significa	qué haces
SUMAR	no se detecta diferencia entre lotes ni repetidas	los lotes cuentan como un build; puedes lanzar otro
AVISO	entre el 10 % y el 30 % del lote ya estaba generado	la celda empieza a agotarse: da su tamaño aproximado
PARAR CELDA	el 30 % o más del lote ya estaba generado	no lances más lotes ahí; si es casi el 100 %, la semilla se repite y es un fallo
NO SUMAR	la configuración difiere, o los embudos difieren más que el azar	no juntes los lotes hasta saber qué cambió

La p del embudo. Es la probabilidad de ver una diferencia así entre lotes por puro azar. Por debajo de 0,01 salta la alarma. Se calcula barajando **islas enteras**, no estrategias: el build es genético y reparte las estrategias en 20 islas, cada una con sus parientes. En el primer lote real, 19 de las 56 supervivientes salieron de una sola isla. Si se tratase cada estrategia como un sorteo independiente, dos lotes honrados parecerían siempre distintos.

«Las islas se dispersan N veces» dice cuánto más varían las islas de un mismo lote de lo que variarían sorteos independientes. Cuanto mayor, menos apuestas de verdad hay detrás de cada cifra.

La tabla por regla enseña qué regla se mueve entre lotes. Sus z son un diagnóstico, no una prueba: con estrategias emparentadas salen más grandes de lo que el azar justifica.

«La celda tiene al menos» es el tamaño del espacio de búsqueda estimado por las repetidas: si de dos lotes de 10.000 se repiten 950, la celda tiene unas 100.000 estrategias distintas. Sin repetidas solo da un suelo.

Un ejemplo completo

La captura de arriba es el ejemplo: dos lotes de 10.000 de doubleROCLong · USDJPY · M30. Sobreviven al paso 8 56 y 48; la p del embudo es 0,74; ninguna estrategia se repite. Decisión: SUMAR, 20.000 generadas acumuladas.

Qué NO te dice

- **No demuestra que los lotes sean iguales**, solo que no ve diferencia. Con dos lotes solo vería diferencias grandes.
- **No ve un error común a todos los lotes**: si todos leyeron los mismos datos malos, salen iguales e igual de equivocados.
- **No ve las casi gemelas**: la huella es exacta, así que dos estrategias con periodo 14 y 15 cuentan como distintas aunque operen casi igual.
- **No cuenta las generadas de verdad**: cada lote son las 10.000 que quedaron en el databank; SQX generó más y sustituyó las peores.

Si algo falla

- `IndexError: list index out of range`: ese proyecto no tiene informe de la puerta (no llegó al paso 8) o no está en el registro de proyectos.
- `FileNotFoundError` con la ruta de una plantilla: la plantilla con la que se creó el lote ya no está donde el registro dice.